

机器视觉技术

孙明竹 刘晓芳

sunmz@nankai.edu.cn

liuxiaofang@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

第二部分 图像处理基础

Image Processing

主要内容

- 第2章 数字图像的概念与性质
 - 2.1 连续图像与数字图像
 - 2.2 数字图像的形成
 - 2.3 数字图像的性质
 - 2.4 图像的数学背景
 - 2.5 图像分析中的数据结构
- 第3章 图像预处理
- 第4章 二值图像处理
- 第5章 传统特征检测

2.1 连续图像与数字图像

➤ 用函数描述图像和信号

- 连续函数：定义域和值域连续
- 离散函数：定义域离散
- 数字函数：定义域和值域离散

➤ 图像 (Image)

- 二维亮度图像是三维场景的透视投影
- 连续图像函数 $f(x, y)$
 - x, y 是平面的坐标，函数值对应于图像点的亮度
- 数字图像 $f(m, n)$
 - 定义域是平面的一个区域， m, n 是有限整数
 - 函数值是有限整数

2.1 连续图像与数字图像

➤ 用矩阵描述数字图像

- 灰度图像中，最低值对应黑，最高值对应白
- 黑白之间的亮度值是灰度阶(gray-level)



156	145	142	140	142	167	180	172	172	158	172	155	135	129	135	140	138	138	141
151	152	161	140	137	169	181	177	175	176	139	87	88	110	122	130	137	143	148
149	173	164	131	136	172	183	187	180	122	58	53	66	69	76	82	97	119	142
167	185	155	129	153	176	188	177	117	86	105	110	98	84	65	39	39	51	77
183	181	135	140	170	193	169	101	88	115	123	110	127	128	102	79	51	40	33
190	166	136	160	194	156	80	76	85	92	100	65	81	128	89	81	73	71	60
188	164	157	188	140	55	52	58	50	40	45	24	29	55	28	28	64	80	84
180	171	187	130	37	35	30	23	25	20	15	17	23	25	17	11	13	47	98
177	190	132	39	30	22	17	18	21	17	17	16	49	114	81	23	11	7	46
194	144	49	40	27	13	24	38	28	28	12	38	61	175	177	114	30	35	44
162	69	62	41	23	13	33	72	47	54	40	50	51	177	197	189	99	51	72
84	79	89	74	63	23	23	79	77	45	43	42	123	200	194	190	134	61	77
81	85	96	105	97	62	40	64	94	90	76	128	187	194	196	165	114	96	79
93	92	98	112	110	103	79	71	76	93	109	128	142	153	151	155	136	117	99
98	103	104	110	110	111	105	107	85	77	67	97	98	89	103	132	144	142	110
103	111	111	115	117	109	112	115	103	100	92	103	106	102	120	126	121	118	114
107	118	122	120	124	125	123	115	112	115	112	115	122	129	136	138	132	138	117
105	117	123	126	135	136	135	138	133	133	133	140	146	146	142	142	139	138	108
104	114	123	126	137	140	138	151	151	144	142	146	150	144	144	147	139	127	109

2.1 连续图像与数字图像

- 用矩阵描述数字图像
 - 彩色图像通过矢量函数描述

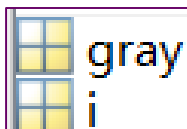


214	211	206	206	204	205	205	217	216	221										
212	206	204	207	203	204	208	218	219	220										
211	199	204	208	204	206	215	220	220	216										
211	192																		
		156	147	138	133	130	135	146	167	173	183								
207	201	156	144	136	133	129	134	149	168	176	183								
200	198	158	139	135	134	133	138	157	171	179	182								
201	204	158	132	142	137	129	127	127	133	140	160	166	174						
203	204	148	133	141	133	127	130	126	132	145	161	169	174						
205	201	136	128	142	128	128	133	131	137	155	166	173	173						
207	206	131	130	142	122	126	130	134	141	161	169	175	170						
		129	130	132	122	132	126	140	149	172	175	172	178						
		133	131	124	120	128	131	156	155	174	178	167	180						
		139	140	123	127	135	147	150	166	170	166	173	172						
				126	131	141	152	165	171	173	175	175	176						
				134	133	146	154	168	167	166	168	172	160						
				138	141	158	162	171	175	174	173	159	115						

2.1 连续图像与数字图像

Matlab

```
i=imread('missrgb.bmp');  
imshow(i);  
gray=rgb2gray(i);  
figure;  
imshow(gray);  
imwrite  
    (gray,'missgray.bmp');
```



512x512 uint8
512x512x3 uint8

2.1 连续图像与数字图像

Matlab

```
i=imread('missrgb.bmp');  
imshow(i);  
gray=rgb2gray(i);  
figure;  
imshow(gray);  
imwrite  
(gray,'missgray.bmp');
```

OpenCV(C++)

```
// 读取图像  
Mat RGBImg = imread("missrgb.bmp");  
// 显示图像  
namedWindow("RGB");  
imshow("RGB", RGBImg);  
  
Mat grayImg;  
cvtColor(RGBImg, grayImg, CV_BGR2GRAY);  
namedWindow("Gray");  
imshow("Gray", grayImg);  
  
waitKey();  
// 保存图像  
imwrite("graymiss.bmp", grayImg);
```


2.1 连续图像与数字图像

Matlab

```
i=imread('missrgb.bmp');  
imshow(i);  
gray=rgb2gray(i);  
figure;  
imshow(gray);  
imwrite  
(gray,'missgray.bmp');
```

```
Data Type: <class 'numpy.ndarray'>  
Shape: (512, 512, 3)  
Data Type of elements: uint8
```

OpenCV(Python)

读取图像

RGBImg = cv2.imread("missrgb.bmp")

显示图像

cv2.namedWindow("RGB")

cv2.imshow("RGB", RGBImg)

将图像转换为灰度图像

grayImg = cv2.cvtColor(RGBImg, cv2.COLOR_BGR2GRAY)

cv2.namedWindow("Gray")

cv2.imshow("Gray", grayImg)

cv2.waitKey(0)

保存灰度图像

cv2.imwrite("graymiss.bmp", grayImg)

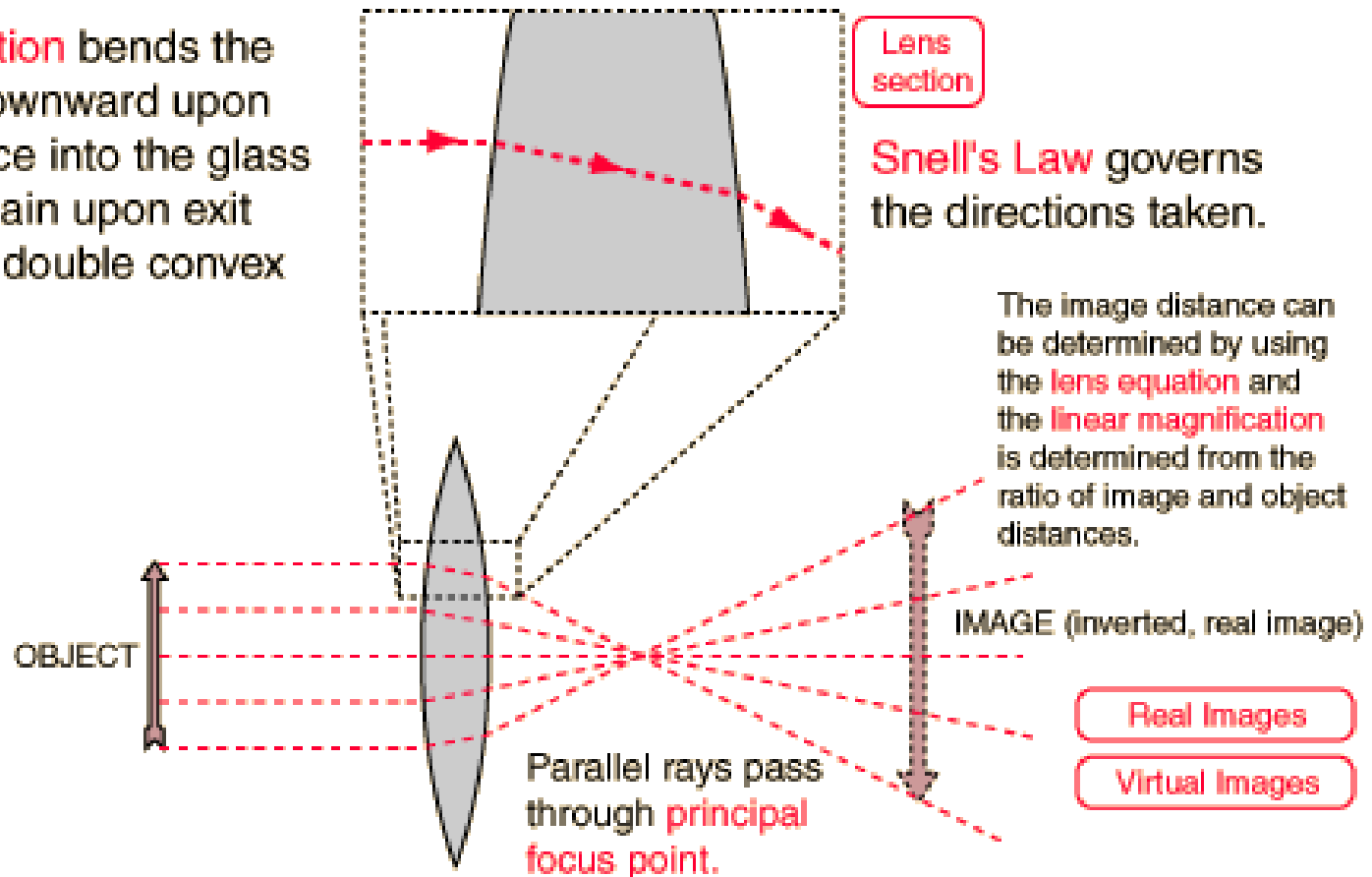
2.2 数字图像的形成

Refraction bends the light downward upon entrance into the glass and again upon exit from a double convex lens.

Lens section

Snell's Law governs the directions taken.

The image distance can be determined by using the **lens equation** and the **linear magnification** is determined from the ratio of image and object distances.

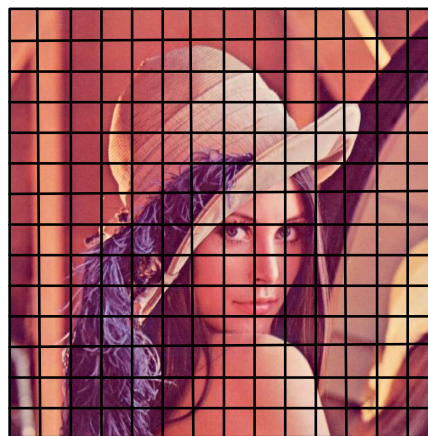
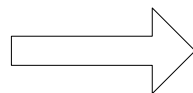


图像数字化

- 传感器获取的图像是平面上的连续函数
- 图像数字化
 - 将连续函数采样(sampled)为M行N列的矩阵
 - 将每个连续样本量化(quantization)为一个整数值，即图像函数的连续范围被分成了K个区间
 - 采样和量化的越精细，即增大M,N,K，连续图像函数的近似性越好

2.2.1 采样

- 连续图像函数 $f(x,y)$ 可通过平面上离散的栅格点来采样
- 图像采样点： $x = m\Delta x, \quad m = 0, 1, \dots, M - 1$
 $y = n\Delta y, \quad n = 0, 1, \dots, N - 1$
- 两个相邻采样点间，在 x 轴和 y 轴上的距离分别是 Δx 和 Δy ，称距离 Δx 和 Δy 为**采样间隔**，采样后得到的矩阵 $f_s(m\Delta x, n\Delta y)$ 构成了离散图像
- 栅格中无限小的采样点对应于数字图像中的像元，即**像素(pixel)**



Matlab

```
i = imread('miss.bmp');
i = i(:,:,1);
imshow(i,[]);
figure;
i2=i(1:2:192,1:2:192);
imshow(i2,[]);
figure;
i3=i(1:4:192,1:4:192);
imshow(i3,[]);
figure;
i4=i(1:8:192,1:8:192);
imshow(i4,[]);
```



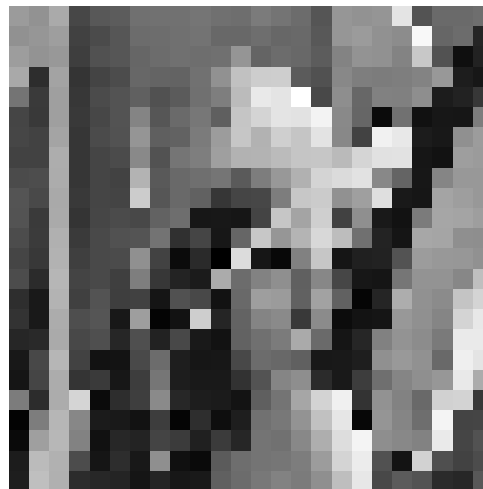
原始图像



2倍采样间隔



4倍采样间隔



8倍采样间隔

OpenCV(C++)

```
int sampleNo = 4; // 采样间隔

int width = img.cols/sampleNo;
int height = img.rows/sampleNo;

Mat sample(height,width,img.type(),Scalar(0));

for(int i=0; i<height; i++)
{
    uchar * imgData = img.ptr<uchar>(i*sampleNo);
    uchar * sampleData = sample.ptr<uchar>(i);
    for(int j=0; j<width; j++)
    {
        sampleData[j] = imgData[j*sampleNo];
    }
}
```

图像: width 矩阵: column

图像: height
矩阵: row

156	145	142	140	142	167	180	172	172	158	172	155	135	129	135	140	138	138	141
151	152	161	140	137	169	181	177	175	176	139	87	88	110	122	130	137	143	148
149	173	164	131	136	172	183	187	180	122	58	53	66	69	76	82	97	119	142
167	185	155	129	153	176	188	177	117	86	105	110	98	84	65	39	39	51	77
183	181	135	140	170	193	169	101	88	115	123	110	127	128	102	79	51	40	33
190	166	136	160	194	156	80	76	85	92	100	65	81	128	89	81	73	71	60
188	164	157	188	140	55	52	58	50	40	45	24	29	55	28	28	64	80	84
180	171	187	130	37	35	30	23	25	20	15	17	23	25	17	11	13	47	98
177	190	132	39	30	22	17	18	21	17	17	16	49	114	81	23	11	7	46
194	144	49	40	27	13	24	38	28	28	12	38	61	175	177	114	30	35	44
162	69	62	41	23	13	33	72	47	54	40	50	51	177	197	189	99	51	72

```

for(int i=0; i<height; i++)
{
    uchar * imgData = img.ptr<uchar>(i*sampleNo);
    uchar * sampleData = sample.ptr<uchar>(i);
    for(int j=0; j<width; j++)
    {
        sampleData[j] = imgData[j*sampleNo];
    }
}

```


OpenCV(Python)

```
import cv2
import numpy as np

i = cv2.imread('miss.bmp')
i = i[:, :, 0]
cv2.imshow('Original Image', i)
i2 = i[0:192:2, 0:192:2]
cv2.imshow('i2', i2)
i3 = i[0:192:4, 0:192:4]
cv2.imshow('i3', i3)
i4 = i[0:192:8, 0:192:8]
cv2.imshow('i4', i4)
```

Matlab

```
i = imread('miss.bmp');
i = i(:,:,1);
imshow(i,[]);
figure;
i2=i(1:2:192,1:2:192);
imshow(i2,[]);
figure;
i3=i(1:4:192,1:4:192);
imshow(i3,[]);
figure;
i4=i(1:8:192,1:8:192);
imshow(i4,[]);
```

2.2.2 量化

- 量化：将采样后的图像函数 $f_s(m\Delta x, n\Delta y)$ 的连续像素值转变为其数字等价量的过程
- 为了使人能够觉察出图像的细微变化，量化的级别要足够高
- 大部分图像数字化设备采用 k 个等间隔的量化方式，如果用 b 位来表示像素亮度的数值，那么亮度级别就是 $k=2^b$
- 通常使用每个像素8位(bit)，特殊的测量设备每个像素使用12位或更多位



256



64



16



2

Matlab

```
i = imread('miss.bmp');
i = double(i(:,:,1));
imshow(uint8(i));
figure;
i2 = floor(i/4)*4;
imshow(uint8(i2));
figure;
i3 = floor(i/16)*16;
imshow(uint8(i3));
figure;
i4 = floor(i/128)*128;
imshow(uint8(i4));
```

OpenCV(C++)

```
const int bit = 8;    // 原图: 256色
int newBit = 4;       // 新图: 2^newBit色
int times = int(pow(2, bit-newBit));

Mat newImg = img.clone();
for(int i=0; i<newImg.rows; i++)
{
    uchar * imgData = newImg.ptr<uchar>(i);
    for(int j=0; j<newImg.cols; j++)
    {
        imgData[j] = int(imgData[j]/times)*times;
    }
}
```

OpenCV(Python)

```
i = cv2.imread('miss.bmp',  
               cv2.IMREAD_GRAYSCALE)  
cv2.imshow('Original Image', i)  
i2 = np.floor(i / 4) * 4  
cv2.imshow('i2', i2.astype(np.uint8))  
i3 = np.floor(i / 16) * 16  
cv2.imshow('i3', i3.astype(np.uint8))  
i4 = np.floor(i / 128) * 128  
cv2.imshow('i4', i4.astype(np.uint8))
```

Matlab

```
i = imread('miss.bmp');  
i = double(i(:,:,1));  
imshow(uint8(i));  
figure;  
i2 = floor(i/4)*4;  
imshow(uint8(i2));  
figure;  
i3 = floor(i/16)*16;  
imshow(uint8(i3));  
figure;  
i4 = floor(i/128)*128;  
imshow(uint8(i4));
```

2.3 数字图像的性质

➤ 2.3.1 数字图像的度量和拓扑性质

➤ 距离(distance)

距离满足同一性、对称性和三角不等式

➤ 同一性

$$D(p, q) \geq 0, \text{ 当且仅当 } p = q \text{ 时, } D(p, q) = 0$$

➤ 对称性

$$D(p, q) = D(q, p)$$

➤ 三角不等式

$$D(p, r) \leq D(p, q) + D(q, r)$$

2.3 数字图像的性质

➤ 2.3.1 数字图像的度量和拓扑性质

➤ 距离(distance)

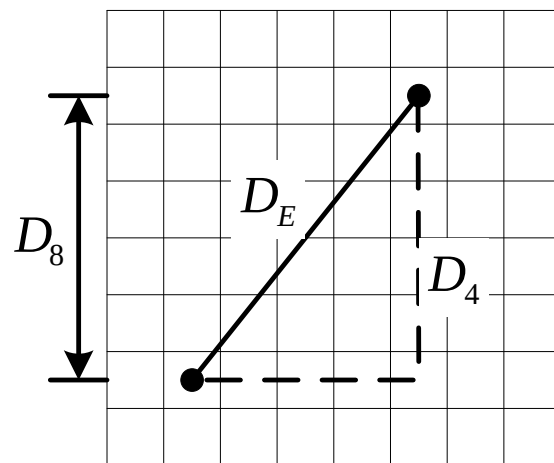
距离满足同一性、对称性和三角不等式

坐标为(m,n)和(h,k)两点之间的距离:

➤ 欧氏距离: $D_E = \sqrt{(m-h)^2 + (n-k)^2}$

➤ 城区距离: $D_4 = |m-h| + |n-k|$

➤ 棋盘距离: $D_8 = \max\{|m-h|, |n-k|\}$

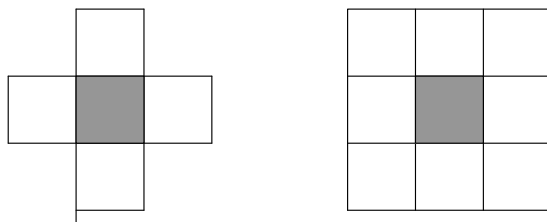


2.3.1 数字图像的度量和拓扑性质



➤ 像素邻接性(adjacency)

- 4-邻接：任意两像素之间的距离为 $D_4 = 1$
- 8-邻接：任意两像素之间的距离为 $D_8 = 1$



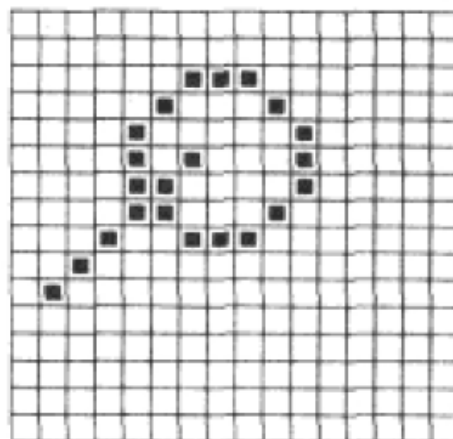
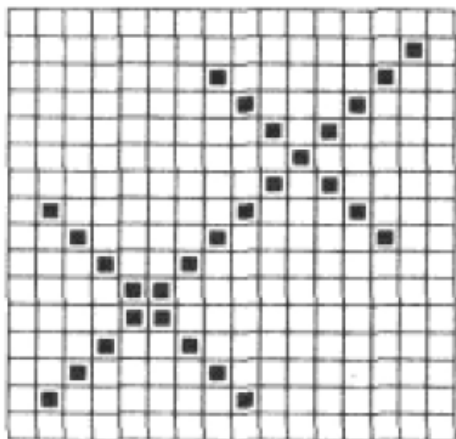
➤ 区域(region):一些彼此邻接的像素组成的重要集合

- 路径：如果定义从像素P到像素Q的路径为一个点序列 $A_1, A_2, \dots, A_n$ ，其中 $A_1 = P$ ， $A_n = Q$ ，且 A_{i+1} 是 A_i 的邻接点， $i = 1, \dots, n - 1$
- 区域：一个集合，集合中任意两个像素之间都存在完全属于该集合的路径

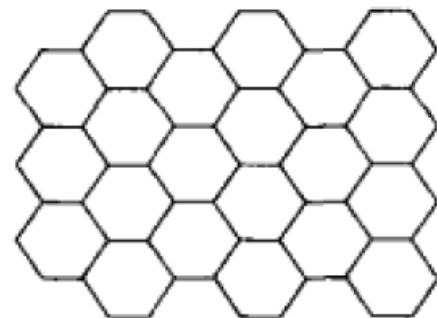
2.3.1 数字图像的度量和拓扑性质



➤ 方形栅格上邻接性与连通性的悖论



➤ 解决悖论的方法：使用六边形栅格

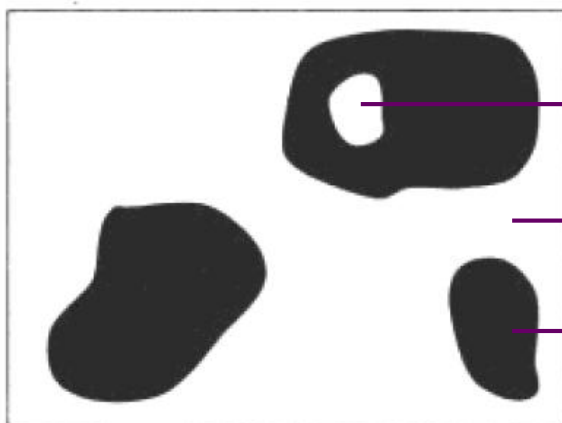


2.3.1 数字图像的度量和拓扑性质



➤ 连通性(contiguous)

- 连通：一幅图像的两个像素之间存在一条路径，这些像素就是连通的，连通关系具有自反性、对称性和传递性
- 区域：彼此连通的像素的集合
- 背景与孔：在图像中定义区域，取区域的补，补集中和边界连通的部分称为背景，其余部分称为孔
- 区域中没有孔，称为简单连通，有孔的区域称为复连通



孔

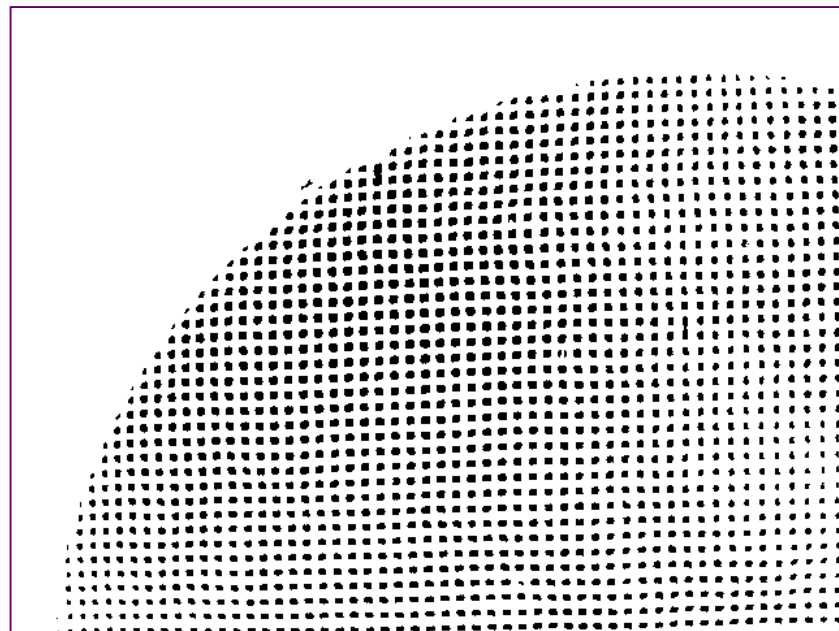
背景

区域

2.3.1 数字图像的度量和拓扑性质



- 举例：区域与分割（Segmentation）

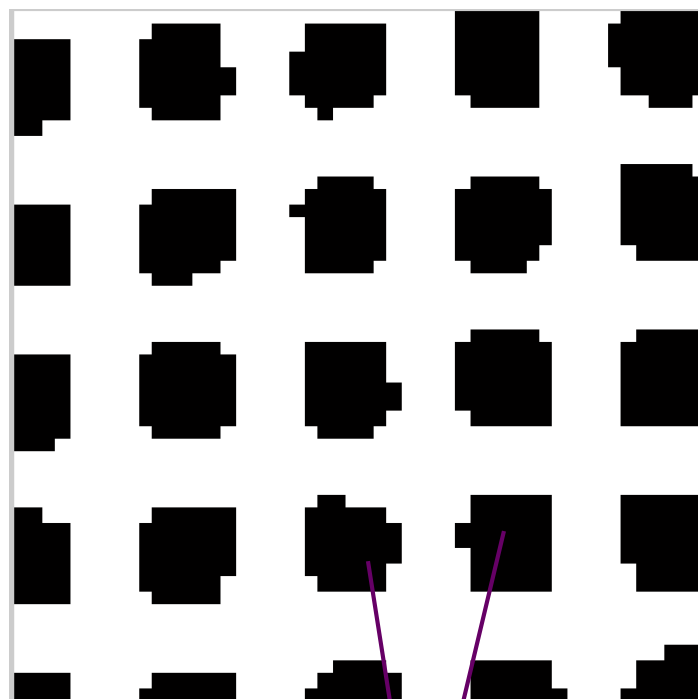
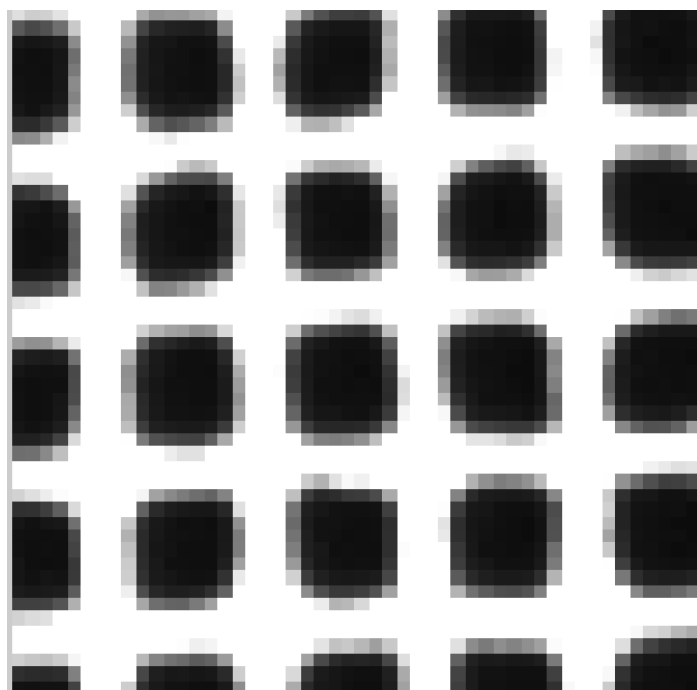


基于图像二值化的分割

2.3.1 数字图像的度量和拓扑性质



- 举例：区域与分割（Segmentation）

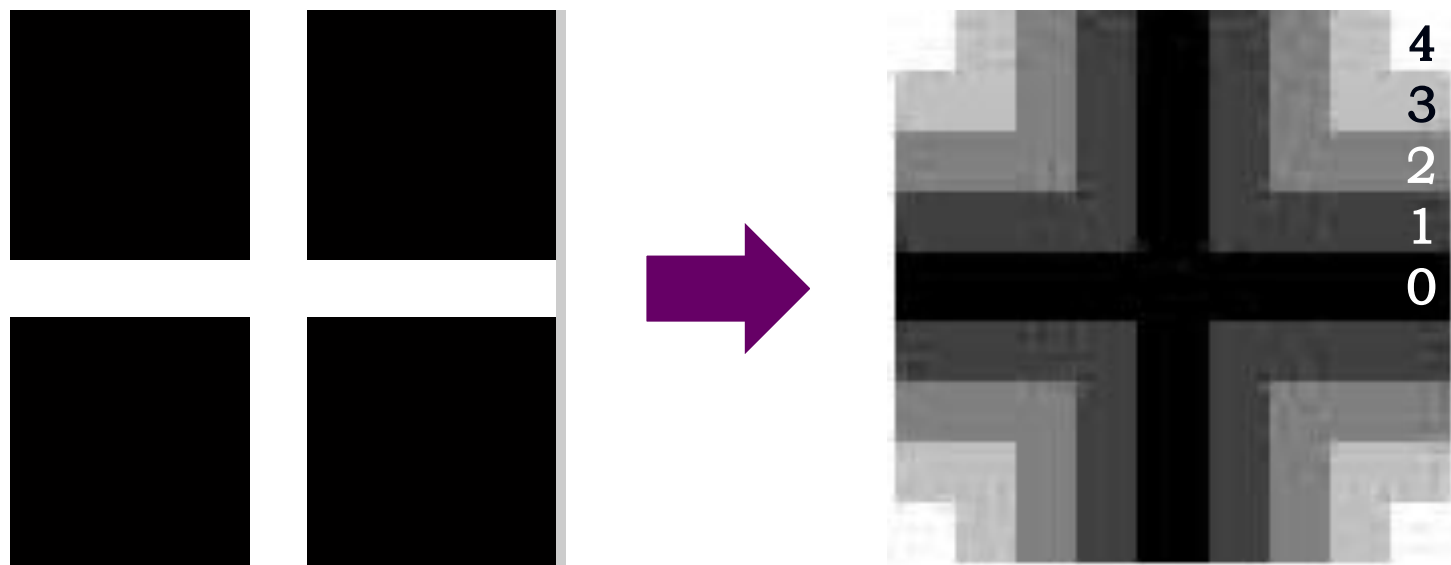


区域/物体

2.3.1 数字图像的度量和拓扑性质



- 距离变换(distance transform)/斜切(chamfering) ➡
 - 距离变换给出图像中的每个像素与某个图像子集的距离
 - 变换结果在子集元素位置处的像素值为0，邻近像素具有较小值，离子集越远数值越大



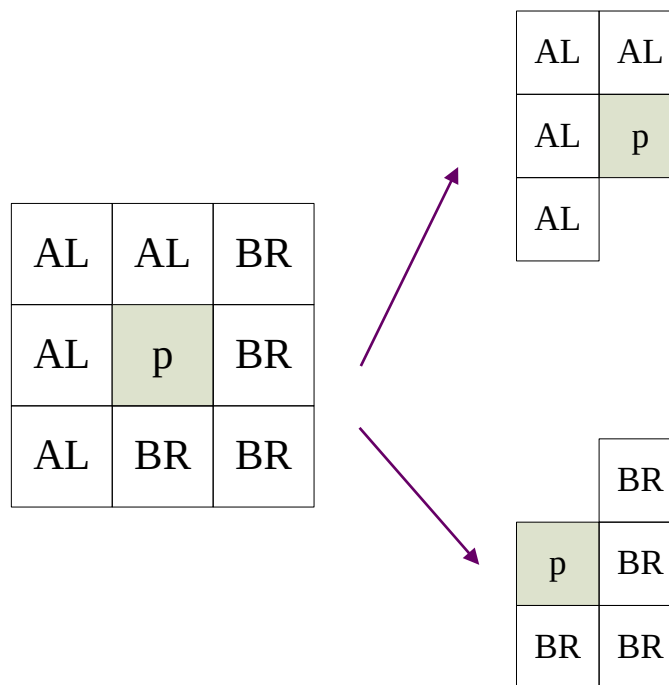
城区距离

2.3.1 数字图像的度量和拓扑性质



➤ 距离变换(distance transform)/斜切(chamfering)

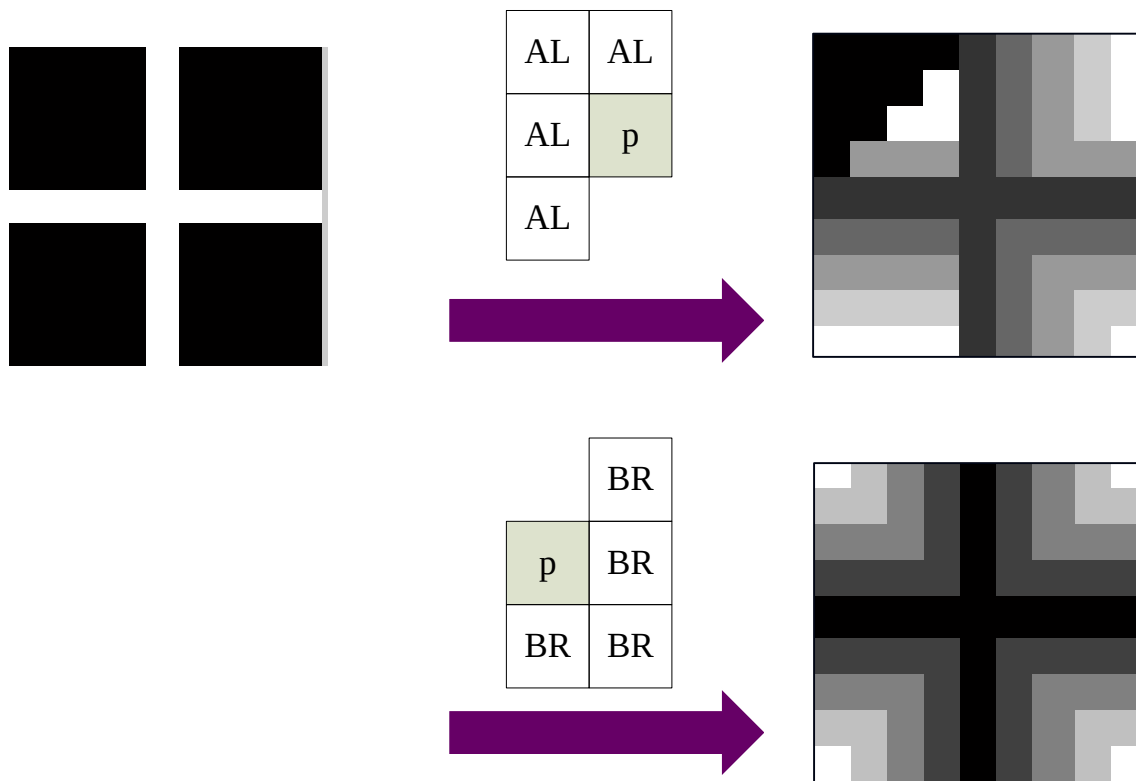
- 设计3*3的图像掩膜，使用掩膜遍历图像
- 把掩膜中心 p 的8邻接像素分成两个掩膜
- 根据掩膜覆盖的图像位置的距离变换结果，计算中心像素 p 的距离变换结果



2.3.1 数字图像的度量和拓扑性质



- 距离变换(distance transform)/斜切(chamfering)
 - 距离变换给出图像中的每个像素与某个图像子集的距离



2.3.1 数字图像的度量和拓扑性质



算法：距离变换/斜切（按照某种距离度量 D ，对大小为 $M \times N$ 的图像的子集 S 计算距离变换）

1. 创建大小为 $M \times N$ 的二维数组 F ，子集 S 元素位置置为0，其他位置置为无穷大
2. 遍历图像，从上到下，从左至右。利用上方和左侧邻接像素(AL 集合)，计算：

$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$

2.3.1 数字图像的度量和拓扑性质



算法：距离变换/斜切（按照某种距离度量D，对大小为M×N的图像的子集S计算距离变换）

1. 创建大小为M×N的二维数组F，子集S元素位置置为0，其他位置置为无穷大
2. 遍历图像，从上到下，从左至右。利用上方和左侧邻接像素(AL集合)，计算：
$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$
3. 遍历图像，从下到上，从右至左。利用下方和右侧邻接像素(BR集合)，计算：
$$F(p) = \min_{q \in BR} [F(p), D(p, q) + F(q)]$$
4. 数组F的结果即为子集S的斜切

$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
		1	∞	0	∞
		∞	∞	0	∞

$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	1	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	1	∞	0	∞	∞
	2	∞	0	∞	∞

∞	∞	0	∞	∞
∞	∞	0	∞	∞
0	0	0	0	0
1	∞	0	∞	∞
2	∞	0	∞	∞

$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

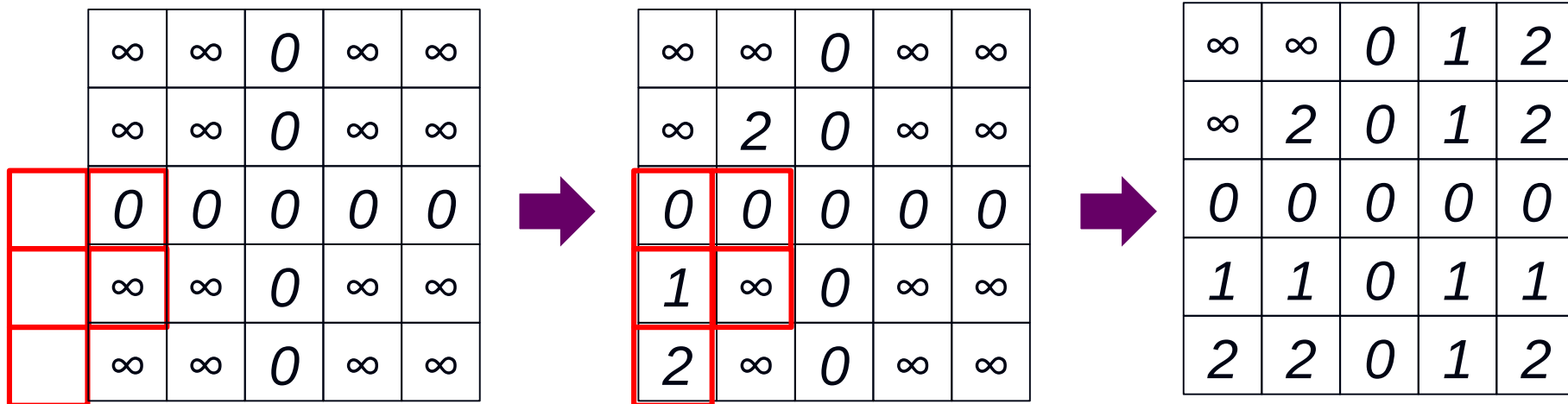
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	∞	∞	0	∞	∞
	∞	∞	0	∞	∞

	∞	∞	0	∞	∞
	∞	∞	0	∞	∞
	0	0	0	0	0
	1	∞	0	∞	∞
	2	∞	0	∞	∞

∞	∞	0	∞	∞
∞	2	0	∞	∞
0	0	0	0	0
1	∞	0	∞	∞
2	∞	0	∞	∞

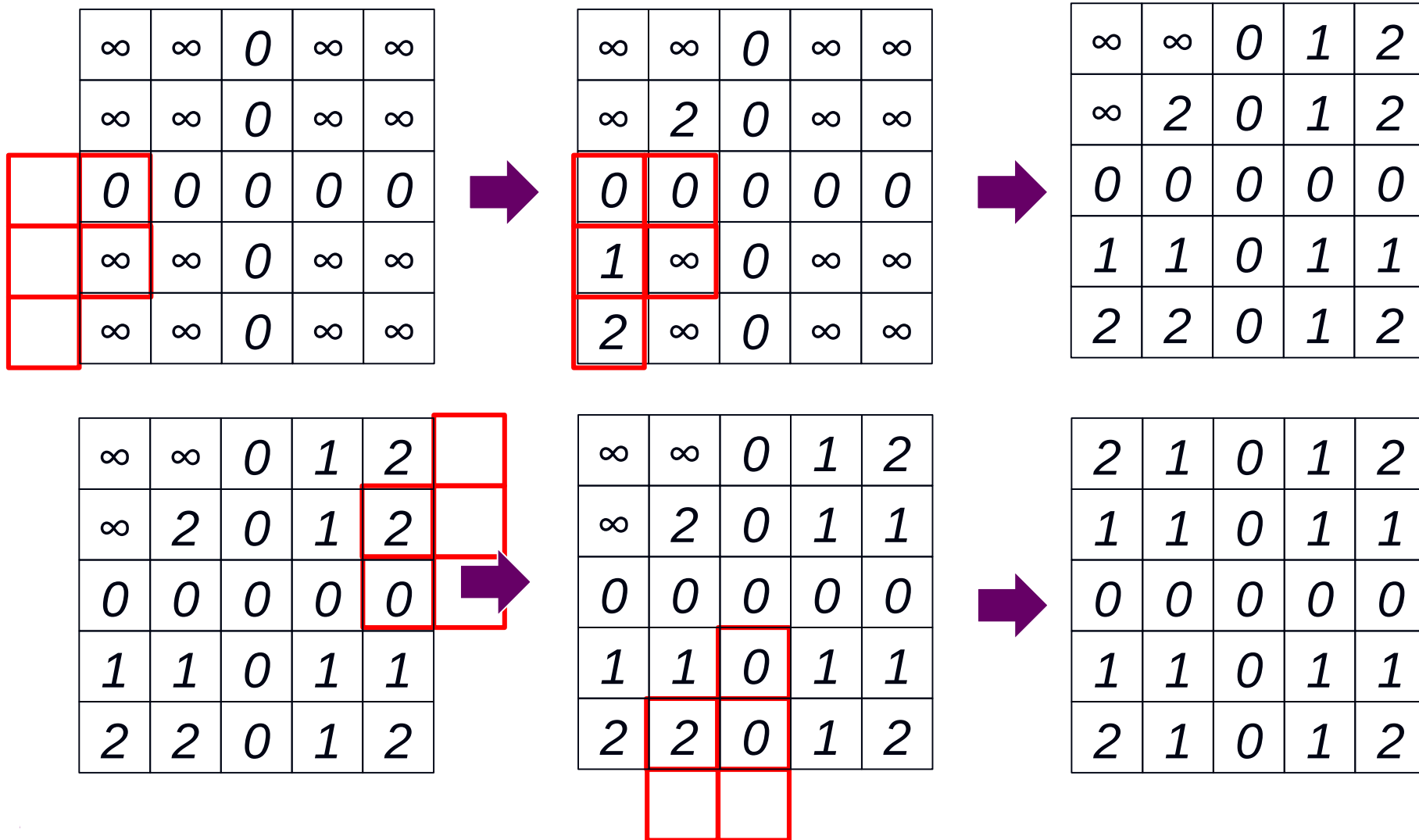
➤ 距离变换示例（城区距离）

$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$



➤ 距离变换示例（城区距离）

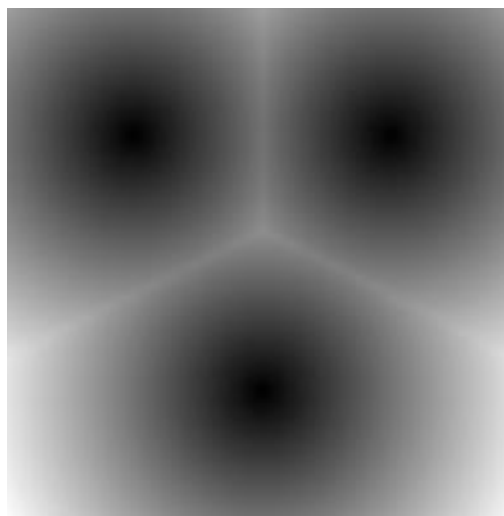
$$F(p) = \min_{q \in BR} [F(p), D(p, q) + F(q)]$$



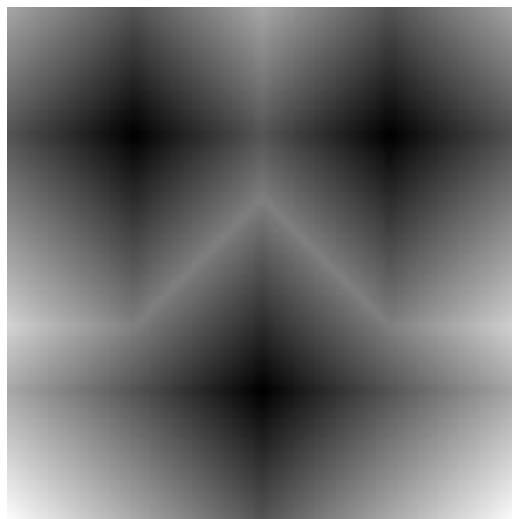
2.3.1 数字图像的度量和拓扑性质



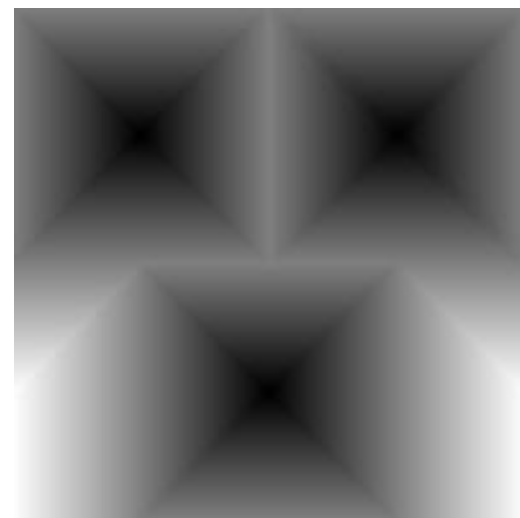
- 距离变换(distance transform)/斜切(chamfering)
- (Matlab:bwdist(I,''))
- (OpenCV:distanceTransform(src, dst, distanceType,...))



欧式距离



城区距离

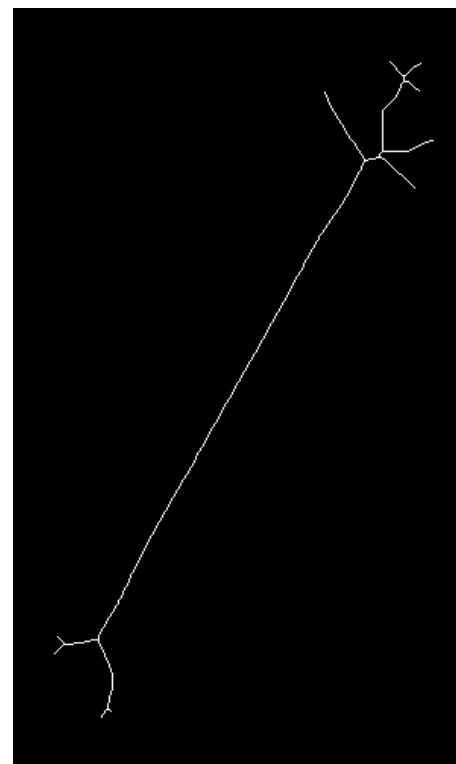
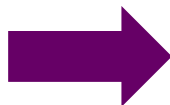
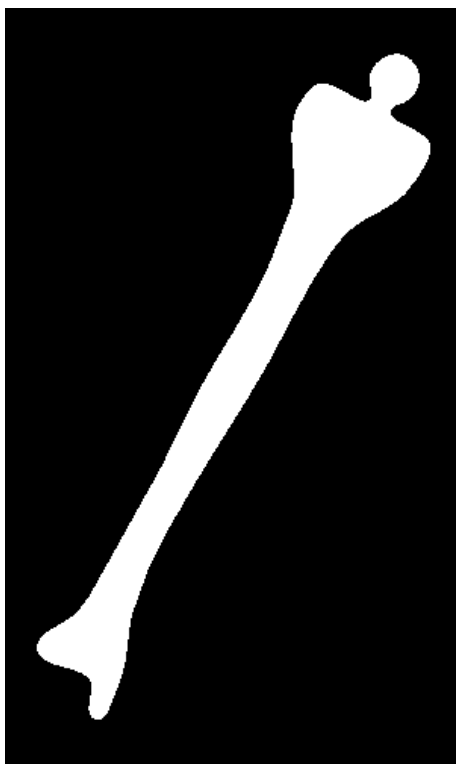


棋盘距离

2.3.1 数字图像的度量和拓扑性质



- 距离变换(distance transform)/斜切(chamfering)
- 举例：形态学骨架提取

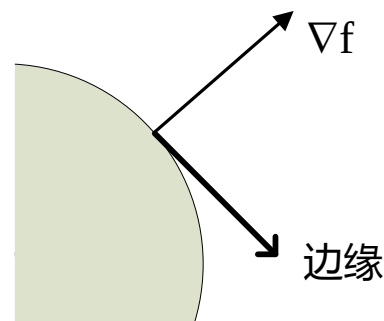


2.3.1 数字图像的度量和拓扑性质



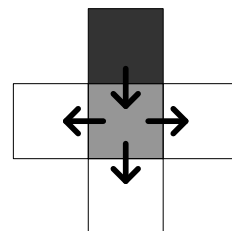
➤ 边缘(edge)

- 边缘是像素与其直接邻域的局部性质
- 边缘是图像上亮度的不连续点，或亮度变化剧烈的地方，它是一个有大小和方向的矢量
- 边缘通过图像梯度表示，边缘的方向与梯度方向垂直，大小与梯度相等



➤ 裂缝边缘(crack edge)

- 在像素间创建的结构，每个像素有4个裂缝边缘
- 裂缝边缘的方向指向亮度增大的方向
- 幅值是相关像素对亮度差的绝对值



2.3.1 数字图像的度量和拓扑性质



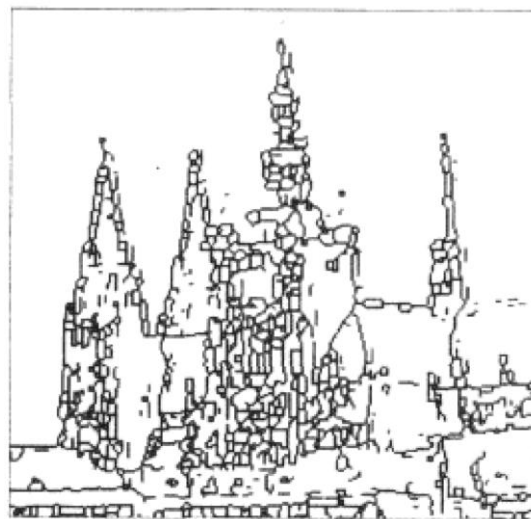
➤ 边缘与裂缝边缘举例



原图



边缘检测



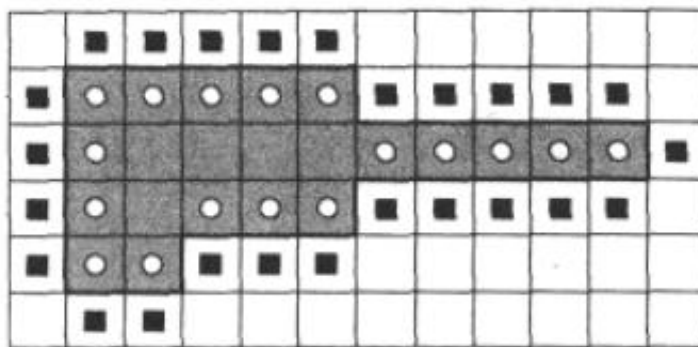
边缘松弛

2.3.1 数字图像的度量和拓扑性质



➤ 边界(border)

- 区域R的边界是其自身的**像素集合**，其中的每个点具有一个或多个R外的邻接点（内部边界）
- 外部边界是区域的背景的边界

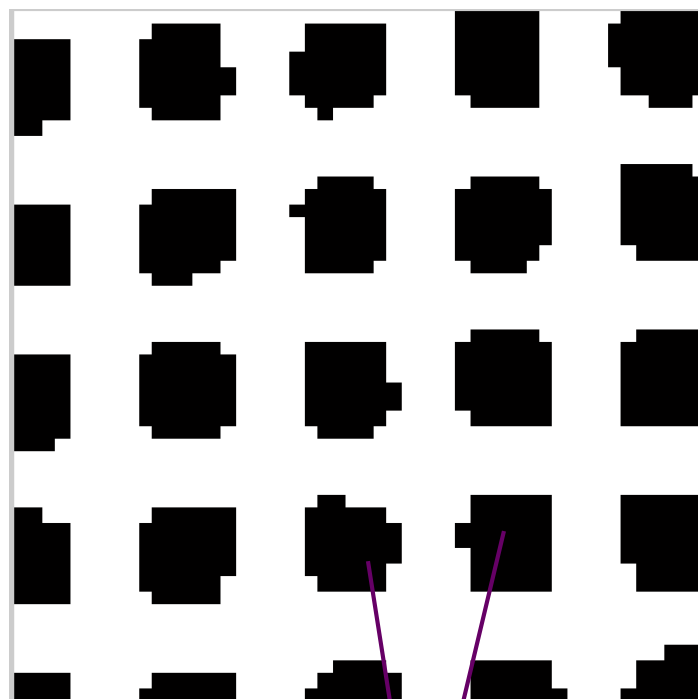
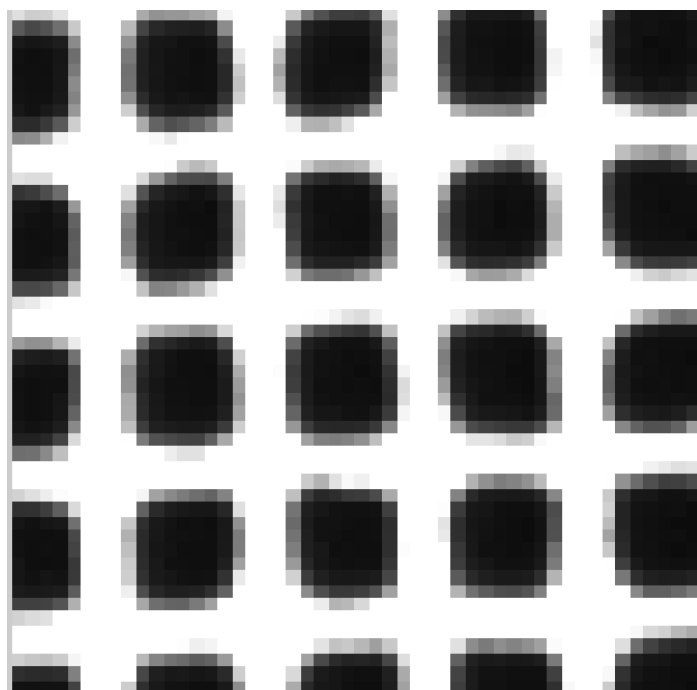


边界和边缘不同，边界是与区域有关的**全局概念**，而边缘表示图像函数的**局部性质**；边界与边缘也相互关联，一种寻找边界的方法是连接显著的边缘

2.3.1 数字图像的度量和拓扑性质



➤ 举例：边缘与边界



区域/物体

2.3.2 直方图(brightness histogram)



- 直方图给出了图像中各个亮度值出现的概率，一幅k阶图像的直方图由具有k个元素的一维数组表示
- 直方图给出了图像的全局信息，可用于像素亮度变换、图像分割

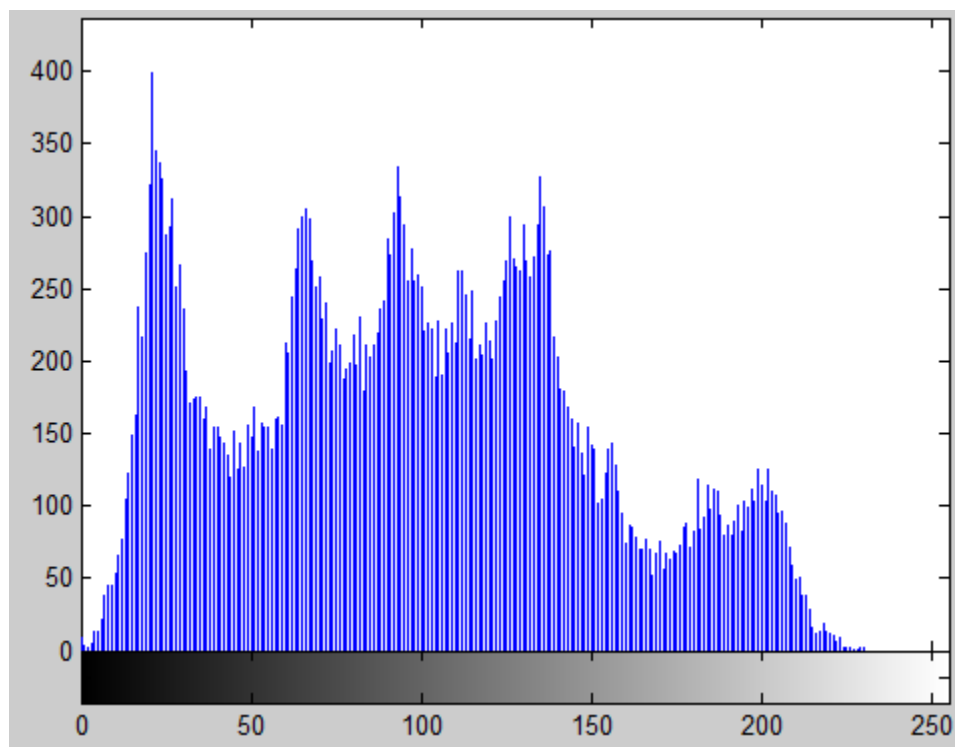
算法：计算亮度直方图

1. 创建大小为k的一维数组H
2. 数组H的所有元素初始化为0
3. 对于图像的每一个元素，做如下处理：

$$H[f(m,n)] = H[f(m,n)] + 1$$

Matlab

```
i=imread('miss.bmp');  
imshow(i);  
figure;  
imhist(i(:,:,1));
```



OpenCV(C++)

```
int channels[] = {0};    // 考察0通道
int histSize[] = {256}; // 直方图中元素数
float range[] = {0,255};
const float * pRange[] = {range};

// 计算直方图,直方图结果放在hist.at<float>(i)
MatND hist;
calcHist(&img, 1, channels, Mat(), hist, 1, histSize, pRange);

// 绘制直方图
double max_val, min_val;    // 直方图数组最大值和最小值
minMaxLoc(hist, &min_val, &max_val, 0, 0);
Mat histImg( cvSize(256*2, 300), CV_8U, Scalar(0));
for(int i=0; i<histSize[0]; i++)
{
    int val = int(hist.at<float>(i) / max_val * 300);
    line(histImg, cvPoint(i*2, 300), cvPoint(i*2, 300-val),
          cvScalar(255, 255, 255), 1);
}
```

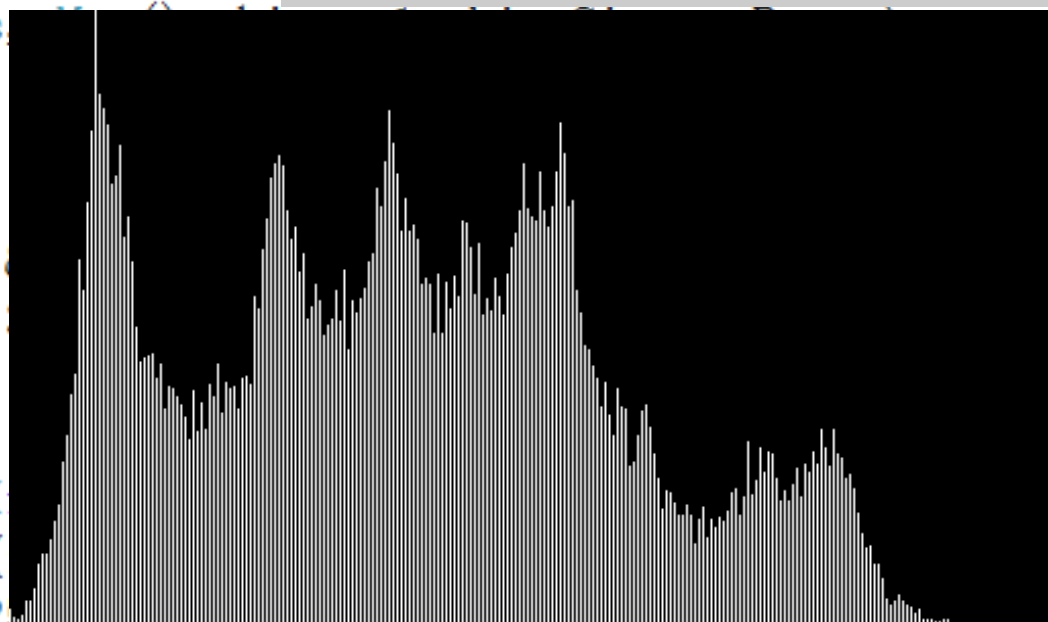
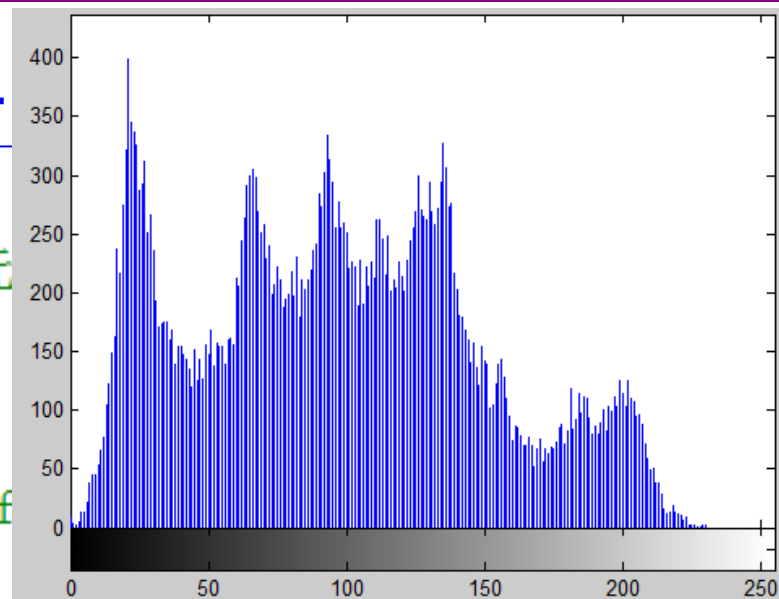

OpenCV(C++

```
int channels[] = {0}; // 考察0通道
int histSize[] = {256}; // 直方图中元
float range[] = {0,255};
const float * pRange[] = {range};
```

```
// 计算直方图,直方图结果放在hist.at<f
MatND hist;
calcHist(&img, 1, channels, hist,
```

```
// 绘制直方图
```

```
double max_val, min_val;
minMaxLoc(hist, &min_val,
Mat histImg( cvSize(256*2,
for(int i=0; i<histSize[0]
{
    int val = int(hist.at<
    line(histImg, cvPoint(
        cvScalar(255,255,255),1,
}
```

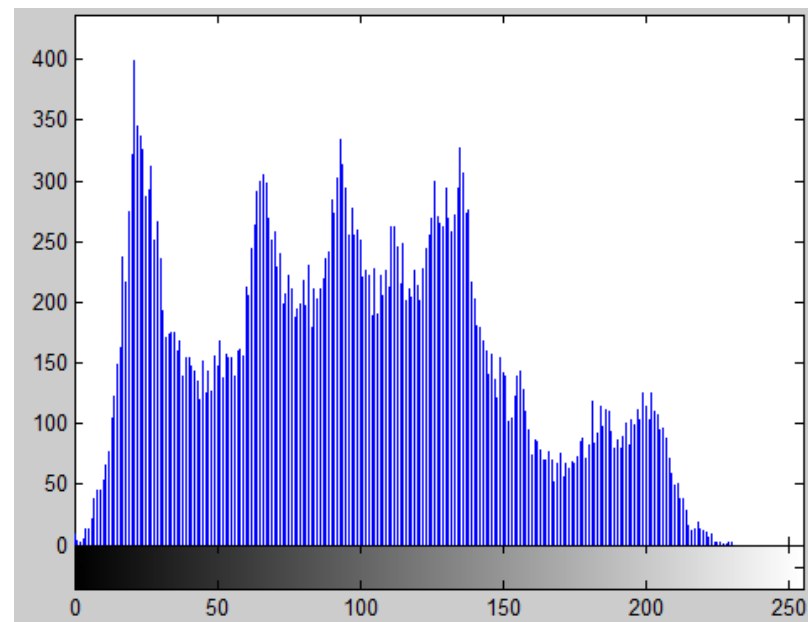
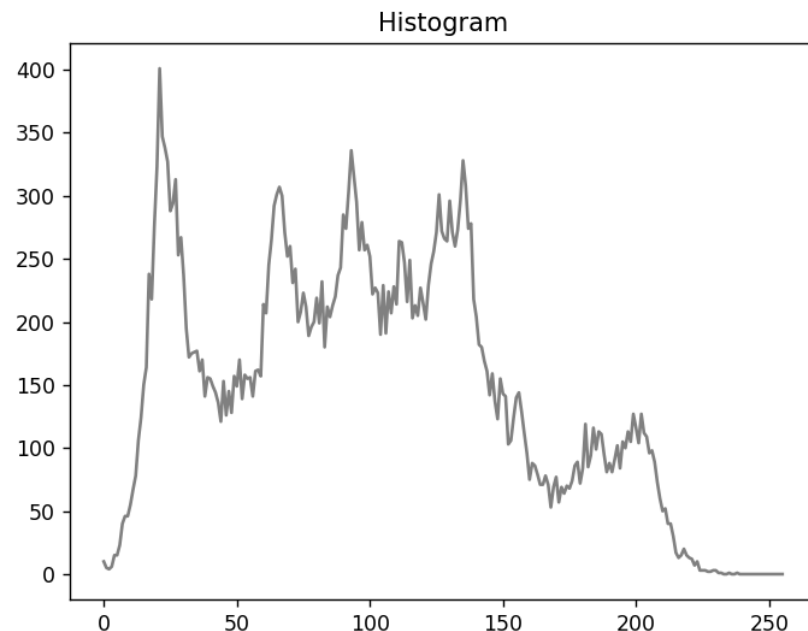


OpenCV(Python)

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('miss.bmp',
                 cv2.IMREAD_GRAYSCALE)

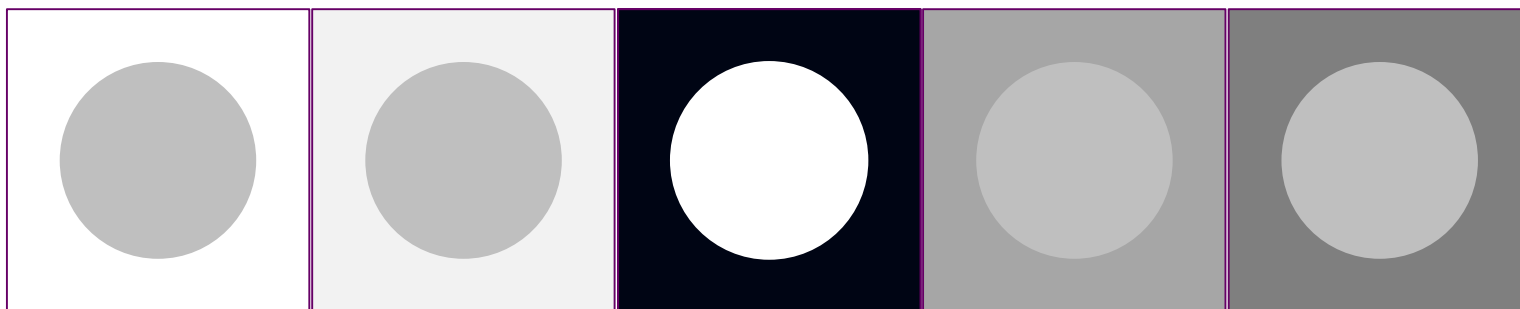
# 计算直方图
hist = cv2.calcHist(
    [img], [0], None, [256], [0, 256])
# 绘制直方图
plt.plot(hist, color='gray')
plt.title('Histogram')
plt.show()
```



2.3.3 图像的视觉感知

➤ 对比度 (contrast)

- 对比度是亮度的局部变化，定义为物体亮度的平均值与背景亮度的比值
- 人的眼睛对亮度的敏感性成对数关系，对于相同的感知，高亮度需要高的对比度
- 条件对比度：表观上的亮度很大程度上取决于局部背景的亮度



2.3.4 图像中的噪声

- 实际图像常受一些随机误差的影响而退化，这些退化称为**噪声**(noise)
- 在图像获取、传输或处理中都可能出现噪声，噪声可能依赖于图像内容，也可能与其无关
- 白噪声(white noise): 具有常量的功率谱，强度不随频率的增加而衰减
- 高斯噪声(Gaussian noise): 是服从正态分布的随机变量，一维情况下的概率密度函数为：

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-u)^2}{2\sigma^2}}$$

2.3.4 图像中的噪声

- 噪声有可能依赖于图像内容，也可能与之无关
 - 加性噪声(additive noise): 噪声与出现的图像信号无关

$$g(m, n) = f(m, n) + \text{noise}(m, n)$$

- 乘性噪声(multiplicative noise): 噪声的幅值与信号本身的幅值相关

$$g = f + \text{noise} \cdot f = f(1 + \text{noise})$$

- 用**信噪比SNR**度量图像的品质，值越大越好

- 计算噪声贡献的平方和: $E = \sum_{(m,n)} \text{noise}^2(m, n)$

- 信号的所有平方和: $F = \sum_{(m,n)} f^2(m, n)$

- 信噪比: $SNR = \frac{F}{E}$

2.3.4 图像中的噪声

➤ 在图像中加入高斯白噪声

➤ Matlab: `imnoise(i, 'gaussian', 0, 0.001)`

➤ OpenCV:

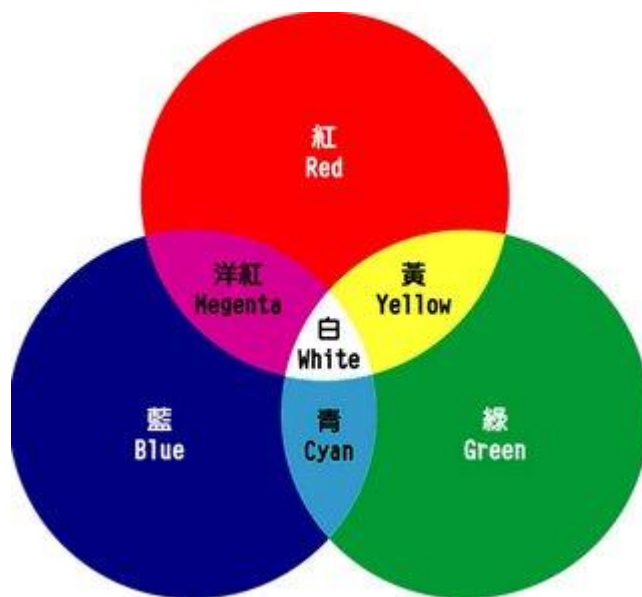
```
// 初始化随机数发生器  
RNG rng = RNG(0xFFFFFFFF);  
// 产生高斯噪声数据  
rng.fill(noise, RNG::NORMAL, 0, 10);  
noise = noise + img;
```



2.3.5 彩色图像

➤ 人类的色彩感知

- 色彩在人类视觉感知中极其重要，色彩与物体反射不同波长的电磁波的能力相关
- 三原色：红(700nm)、绿(546.1nm)、蓝(438.5nm)



彩色图像的色彩模型 (I)

➤ RGB模型

- 红(Red)、绿(Green)、蓝(Blue)
- 一般计算机硬件使用的色彩模型
- 一个像素与一个三维向量(r,g,b)相关联, 分量分别对应于相应色彩的亮度
- 用k表示每个原色的量化粒度 (通常 $k=256$) , 则
(0,0,0):黑 ($k-1,k-1,k-1$):白 ($k-1,0,0$):纯红
- 共有 $k \times k \times k$ 种不同颜色的色彩空间

➤ CMYK模型

- 青(Cyan)、品红(Magenta)、黄(yellow)、黑(Black)
- 基于次生颜色, 在彩色印刷设备中用于墨水组合

彩色图像的色彩模型 (I)

➤ RGB模型

- 红(Red)、绿(Green)、蓝(Blue)
- 一般计算机硬件使用的色彩模型
- 一个像素与一个三维向量(r,g,b)相关于相应色彩的亮度
- 用k表示每个原色的量化粒度 (通常 (0,0,0):黑 (k-1,k-1,k-1):白 (k-1,k-1,k-1):白 (k-1,k-1,k-1):白)
- 共有 $k \times k \times k$ 种不同颜色的色彩空间



➤ CMYK模型

- 青(Cyan)、品红(Magenta)、黄(yellow)、黑(Black)
- 基于次生颜色，在彩色印刷设备中用于墨水组合



彩色图像



红色分量



绿色分量



蓝色分量

Matlab

```
i=imread('missrgb.bmp');
imshow(i);
figure; imshow(i(:,:,1));
figure; imshow(i(:,:,2));
figure; imshow(i(:,:,3));
```

OpenCV(C++)

```
// 使用split函数
Mat matBGR[3];
// matBGR的三个分量
// 色彩排列顺序：B、G、R
split(img, matBGR);
```

OpenCV(Python)

```
# 读取图像
i = cv2.imread('missrgb.bmp')

# 显示原始图像
cv2.imshow('Original Image', i)

# 分别显示每个通道
# 默认使用 BGR 格式
cv2.imshow('Blue', i[:, :, 0]) # B通道
cv2.imshow('Green', i[:, :, 1]) # G通道
cv2.imshow('Red', i[:, :, 2]) # R通道
```

Matlab

```
i=imread('missrgb.bmp');
imshow(i);
figure; imshow(i(:,:,1));
figure; imshow(i(:,:,2));
figure; imshow(i(:,:,3));
```

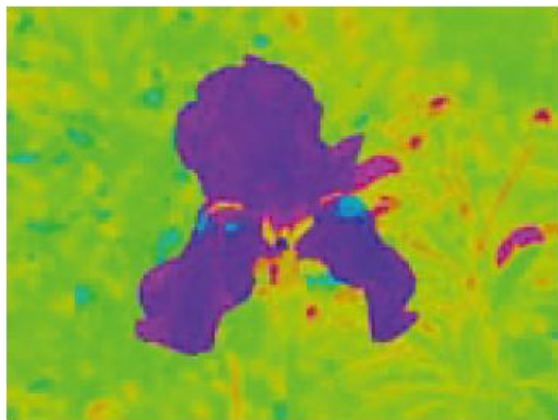
OpenCV(C++)

```
// 使用split函数
Mat matBGR[3];
// matBGR的三个分量
// 色彩排列顺序：B、G、R
split(img, matBGR);
```

彩色图像的色彩模型 (II)

➤ HSB/HSV模型

- 色调(Hue): 表示感知到的色彩, 主要指光的波长
- 色饱和度(Saturation): 表示色彩被白光冲淡的程度
- 亮度 (Brightness) : 表示色彩的明暗程度
- 从人类的视觉系统出发, 在开发图像处理算法中很有用



H



S

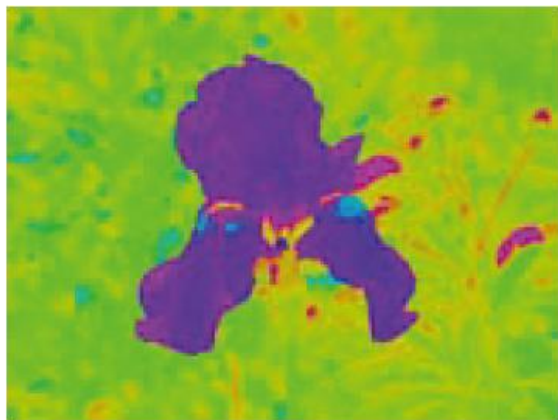


V

彩色图像的色彩模型 (II)

➤ HSB/HSV模型

- 色调(Hue): 表示感知到的色彩, 主要
- 色饱和度(Saturation): 表示色彩被白
- 亮度 (Brightness): 表示色彩的明暗和
- 从人类的视觉系统出发, 在开发图像处理算法中很有用



H



S



V

彩色图像的色彩模型 (II)

➤ HSB/HSV模型

- 色调(Hue): 表示感知到的色彩, 主要指光的波长
- 色饱和度(Saturation): 表示色彩被白光冲淡的程度
- 亮度 (Brightness) : 表示色彩的明暗程度
- 从人类的视觉系统出发, 在开发图像处理算法中很有用

➤ YIQ模型与YUV模型

- 用于彩色广播电视, Y分量表示亮度; IQ/UV分量携带颜色信息
- YIQ与RGB之间的变换:

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.275 & -0.321 \\ 0.212 & -0.523 & 0.311 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

彩色图像与灰度图像

- 变换公式: $Y = 0.299R + 0.587G + 0.114B$
- (Matlab:rgb2gray)
- (OpenCV:cvtColor(, , COLOR_BGR2GRAY))

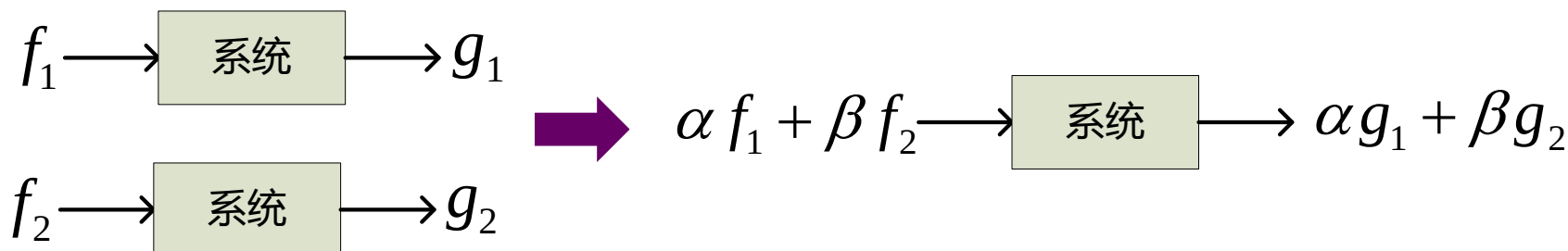




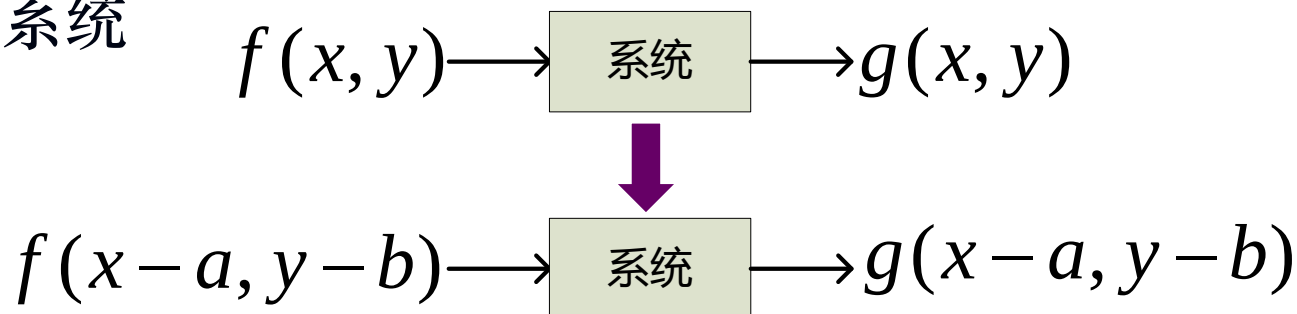
2.4 图像的数学背景

➤ 2.4.1 线性移不变系统

➤ 线性系统

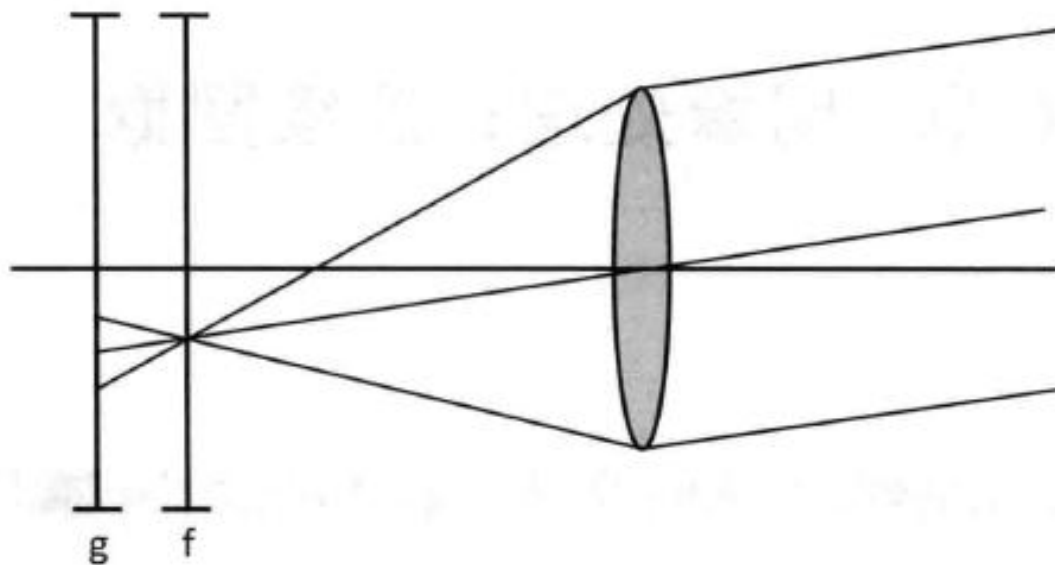


➤ 移不变系统



2.4.1 线性移不变系统

- 举例：从理想聚焦图像 $f(x, y)$ 到失焦图像 $g(x, y)$



- 成像系统的缺陷常被看作一个线性移不变系统
- 线性移不变系统将理想图像变成我们观察的图像

2.4.2 卷积与点扩散函数

➤ 狄拉克分布(Dirac distribution) / 单位冲击函数

➤ $\delta(x, y)$ 满足

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \delta(x, y) dx dy = 1$$

对于所有 $x, y \neq 0$, 有 $\delta(x, y) = 0$

➤ 狄拉克函数的筛选性

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) \delta(x - \lambda, y - \mu) dx dy = f(\lambda, \mu)$$

该公式描述了连续图像函数 $f(x, y)$ 的采样过程

2.4.2 卷积与点扩散函数

➤ 卷积(Convolution)的定义

二维函数 $f(x, y)$ 和 $h(x, y)$ 的卷积 $g(x, y)$ 记为 $f * h$ ，由积分定义：

$$\begin{aligned} g(x, y) &= f(x, y) * h(x, y) \\ &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x-a, y-b)h(a, b)dad b \\ &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(a, b)h(x-a, y-b)dad b \end{aligned}$$

➤ 卷积的性质

- 交换率 $f * h = h * f$
- 结合律 $(f * h_1) * h_2 = f * (h_1 * h_2)$
- 微分交换 $\frac{d}{dx}(f * h) = \frac{df}{dx} * h = f * \frac{dh}{dx}$



2.4.2 卷积与点扩散函数

➤ 卷积与线性移不变系统

➤ 卷积是线性移不变操作

$$g(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(a, b) h(x - a, y - b) da db$$

- 对于一个系统，对于给定的输入 $f(x, y)$ ，若系统响应（系统输出） $g(x, y)$ 可以表示成**卷积的形式**，则该系统是一个**线性移不变系统**



- 逆命题成立：任意线性移不变系统都可以表示成卷积

2.4.2 卷积与点扩散函数

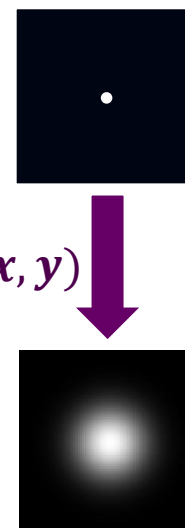
➤ 点扩散函数(Point Spread Function)

- 根据狄拉克分布的定义：

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \delta(x, y) h(x, y) dx dy = h(0, 0)$$

换元得到 $\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \delta(x - a, y - b) h(a, b) da db = \boxed{h(x, y)}$

对比卷积定义 $\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x - a, y - b) h(a, b) da db = g(x, y)$

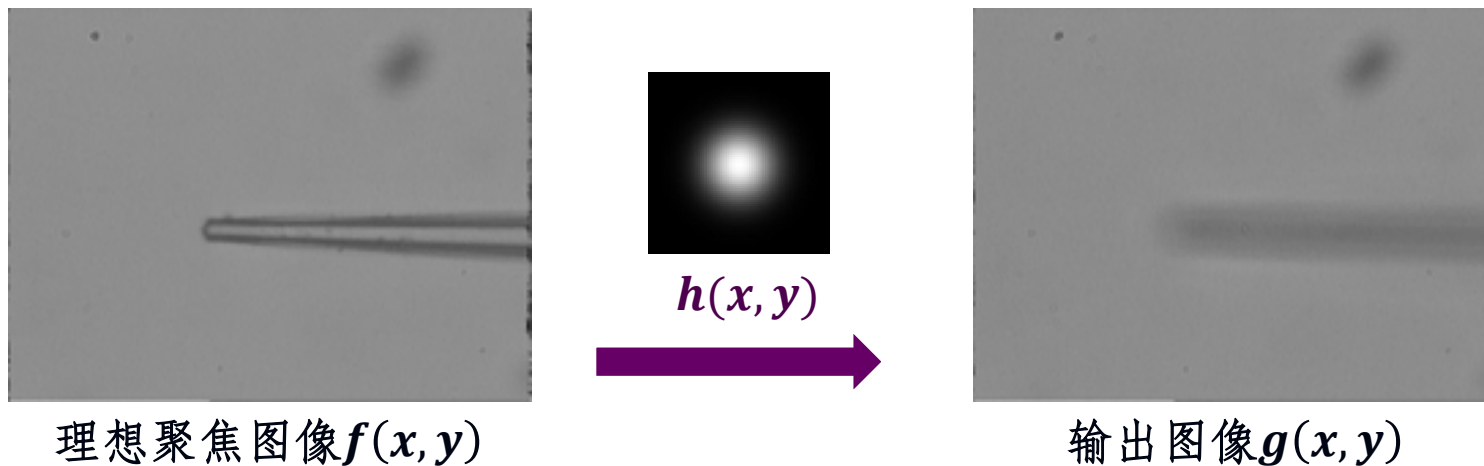


- $h(x, y)$ 的物理意义： $h(x, y)$ 是当系统的输入是狄拉克分布 $\delta(x, y)$ 时，系统所产生的响应

2.4.2 卷积与点扩散函数

➤ 点扩散函数

- 举例：从理想聚焦图像 $f(x, y)$ 到失焦图像 $g(x, y)$



- 点扩散参数表述了一个线性移不变系统的全部特征

2.4.2 卷积与点扩散函数

➤ 二维离散卷积 $g(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x-a, y-b)h(a, b)dad b$ ➡

(Matlab:conv2)



$$g(m, n) = \sum_{k=0}^{M_2-1} \sum_{l=0}^{N_2-1} f(m-k, n-l)h(k, l)$$

$$= \sum_{k=0}^{M_1-1} \sum_{l=0}^{N_1-1} f(k, l)h(m-k, n-l)$$

$f(x, y)$ 尺寸: $M_1 \times N_1$

$h(x, y)$ 尺寸: $M_2 \times N_2$

$m = 0, 1, \dots, M_1 + M_2 - 2;$

$n = 0, 1, \dots, N_1 + N_2 - 2;$

- h : 卷积核 / 卷积掩模 / 窗口
- 手工计算离散卷积的步骤
 - 反转、相乘、相加

➤ 举例：手工计算一维离散卷积

$$g(m) = \sum_{k=0}^{M_1-1} f(k)h(m-k) \quad \begin{array}{l} f(x) \text{ 尺寸: } M_1, h(x) \text{ 尺寸: } M_2 \\ m = 0, 1, \dots, M_1 + M_2 - 2 \end{array}$$

$$f = [1 \ 2 \ 3 \ 4 \ 5]^T, h = [1 \ 2 \ 3]^T \quad M_1 = 5, M_2 = 3 \quad m = 0, 1, \dots, 6$$

3	2	1	0	0	0	0	0	0			
		1	2	3	4	5		3	2	1	

$$m = 0, g(0) = f(0) * h(0) = 1$$

$$m = 1, g(1) = f(0) * h(1) + f(1) * h(0) = 4$$

$$m = 2, g(2) = f(0) * h(2) + f(1) * h(1) + f(2) * h(0) = 10$$

$$m = 3, g(3) = f(1) * h(2) + f(2) * h(1) + f(3) * h(0) = 16$$

$$m = 4, g(4) = f(2) * h(2) + f(3) * h(1) + f(4) * h(0) = 22$$

$$m = 5, g(5) = f(3) * h(2) + f(4) * h(1) = 22$$

$$m = 6, g(6) = f(4) * h = 15$$

2.4.2 卷积与点扩散函数

➤ 原始图像与Prewitt算子 h_1 做卷积



$$h_1 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



136	133	134	135	130
132	132	130	133	131
131	132	125	131	131
131	130	130	132	129
130	131	129	130	130

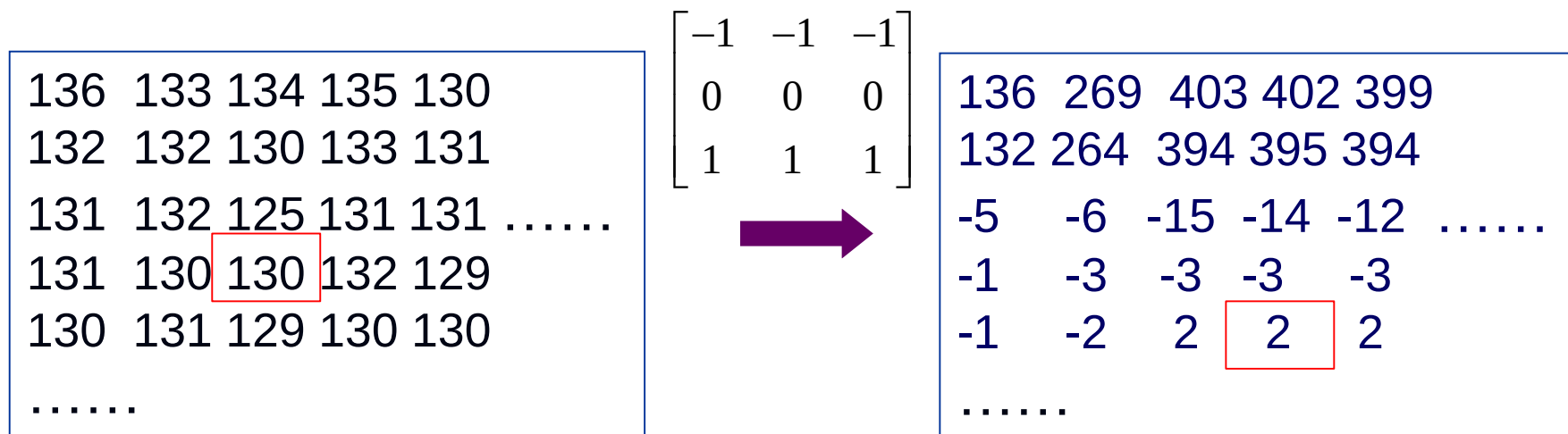
$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$



136	269	403	402	399
132	264	394	395	394
-5	-6	-15	-14	-12
-1	-3	-3	-3	-3
-1	-2	2	2	2

2.4.2 卷积与点扩散函数

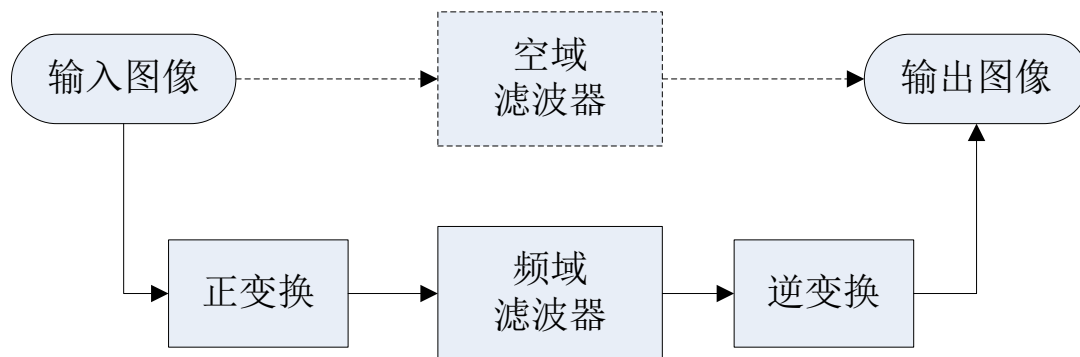
➤ 原始图像与Prewitt算子h1做卷积



注意：区别原始卷积（图像尺寸增大）、图像滤波中的卷积（图像尺寸不变）、深度学习中的卷积（操作更加灵活）

2.4.3 傅里叶变换与卷积定理

➤ 积分线性变换



➤ 傅里叶变换(Fourier Transform)

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-2\pi i(xu + yv)} dx dy$$

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{2\pi i(xu + yv)} du dv$$

$x - y$ 平面表示空间域, $u - v$ 平面表示频率域

2.4.3 傅里叶变换与卷积定理

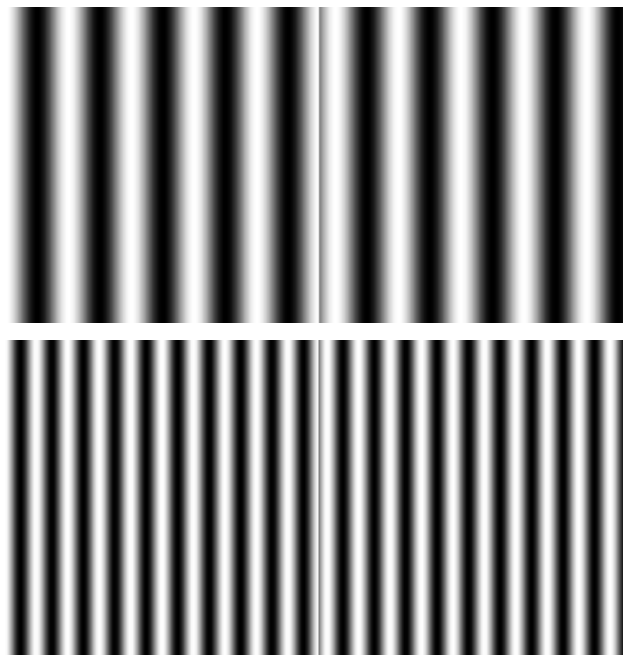
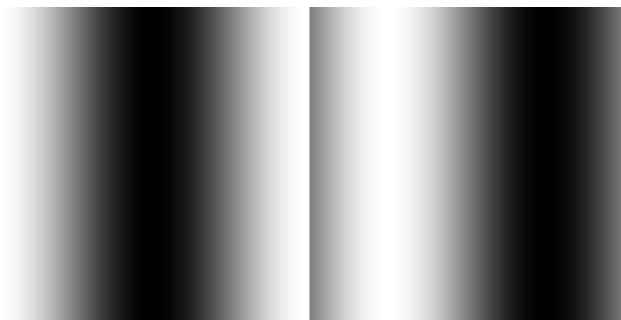
➤ 傅里叶变换(Fourier Transform)

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{2\pi i(xu + yv)} du dv$$

↑

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) \left(\cos(2\pi(xu + yv)) + i \sin(2\pi(xu + yv)) \right) du dv$$

➤ $u=0, v=1, 5, 10$



2.4.3 傅里叶变换与卷积定理

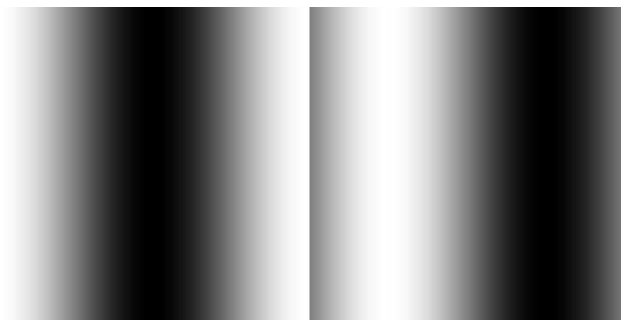
➤ 傅里叶变换(Fourier Transform)

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{2\pi i(xu + yv)} du dv$$

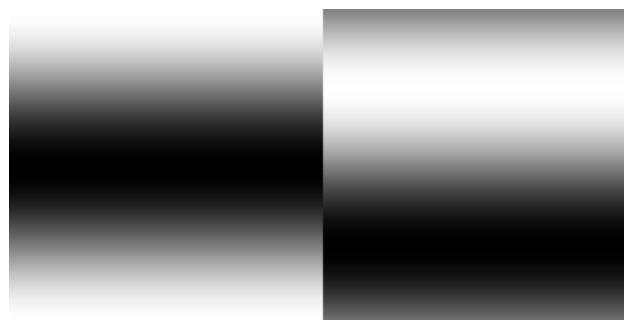
↑

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) \left(\cos(2\pi(xu + yv)) + i \sin(2\pi(xu + yv)) \right) du dv$$

➤ $u=0, v=1$



$v=0, u=1$



2.4.3 傅里叶变换与卷积定理

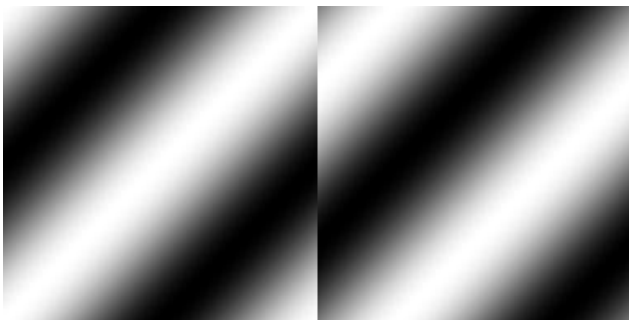
➤ 傅里叶变换(Fourier Transform)

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{2\pi i(xu + yv)} du dv$$

↑

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \boxed{F(u, v)} (\cos(2\pi(xu + yv)) + i \sin(2\pi(xu + yv))) du dv$$

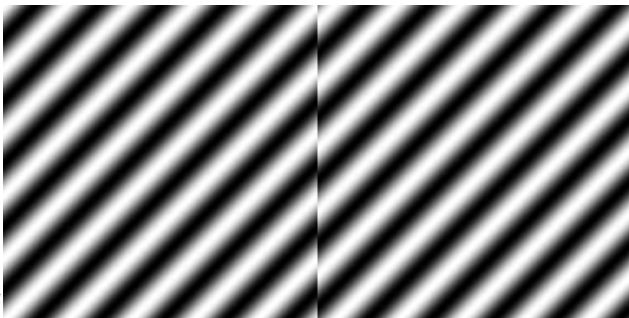
u=1
v=1



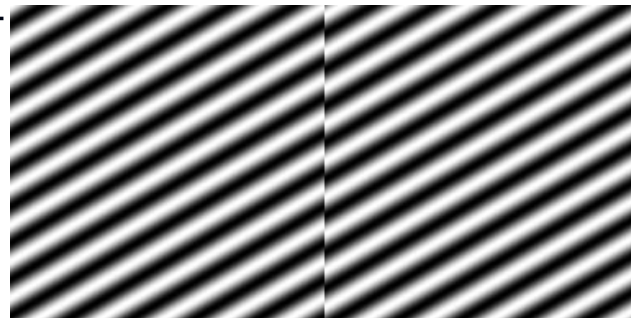
u=5
v=5*√3



u=5
v=5



u=5*√3
v=5



2.4.3 傅里叶变换与卷积定理

➤ 傅里叶变换(Fourier Transform)

- 用 F 表示傅立叶变换算子，公式简写为：

$$F(u, v) = F\{f(x, y)\}$$

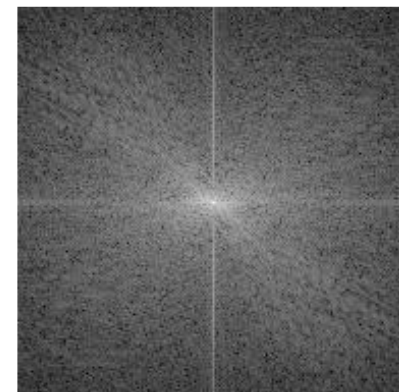
傅立叶变换的结果 $F(u, v)$ 通常为复数，令 $R(u, v)$ 和 $I(u, v)$ 分别表示 $F(u, v)$ 的实部和虚部

- 傅立叶频谱： $|F(u, v)| = [R^2(u, v) + I^2(u, v)]^{1/2}$

- 相位角： $\phi(u, v) = \arctan\left[\frac{I(u, v)}{R(u, v)}\right]$



原始图像

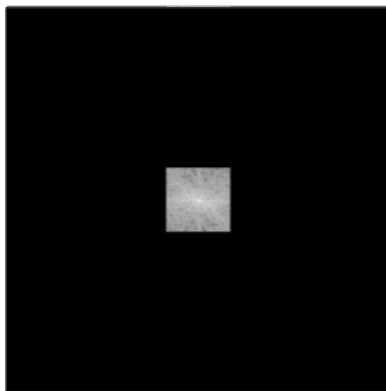


对数显示

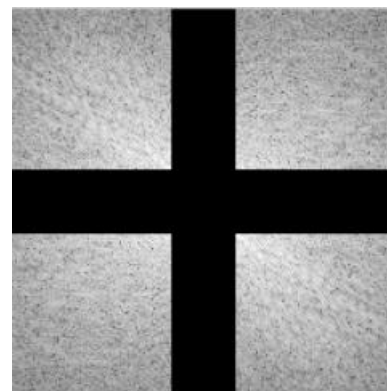
2.4.3 傅里叶变换与卷积定理



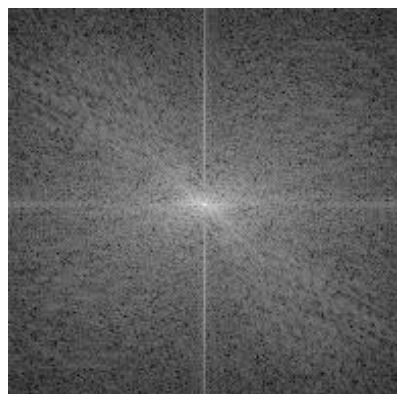
原始图像



低频区域



高频区域

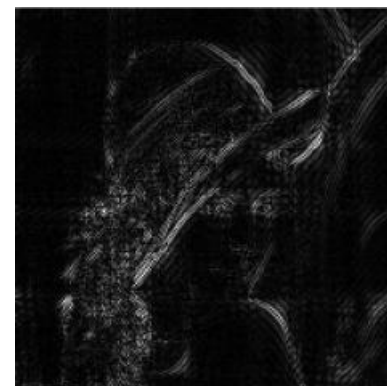


傅里叶变换对数显示

机器人与信息自动化研究所

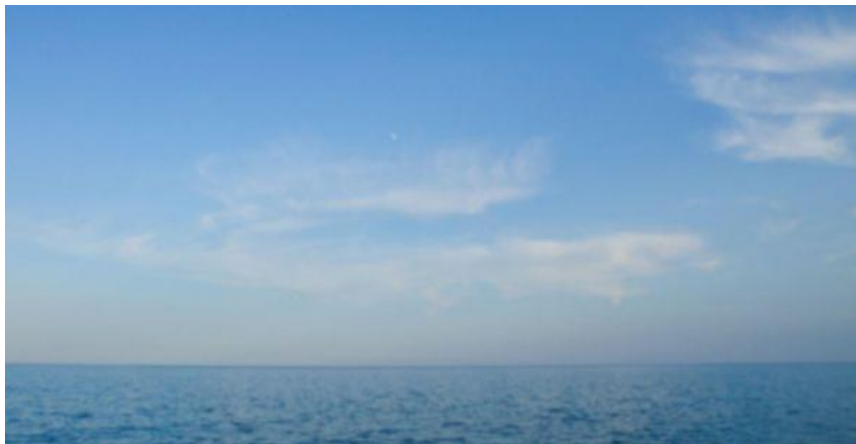


傅里叶反变换结果

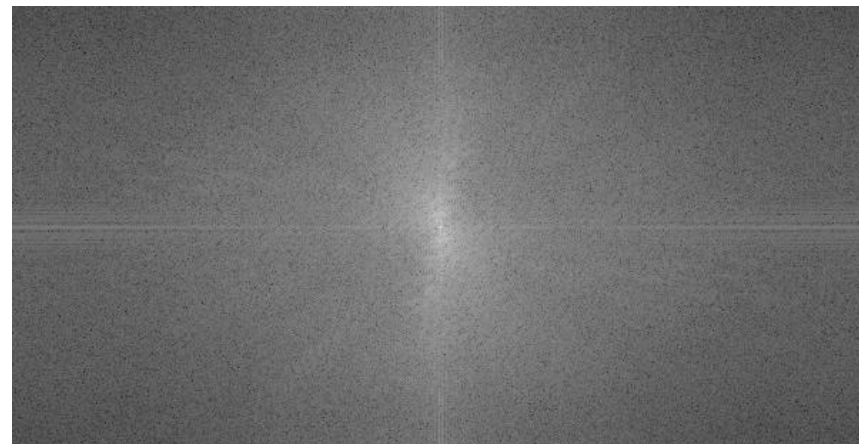
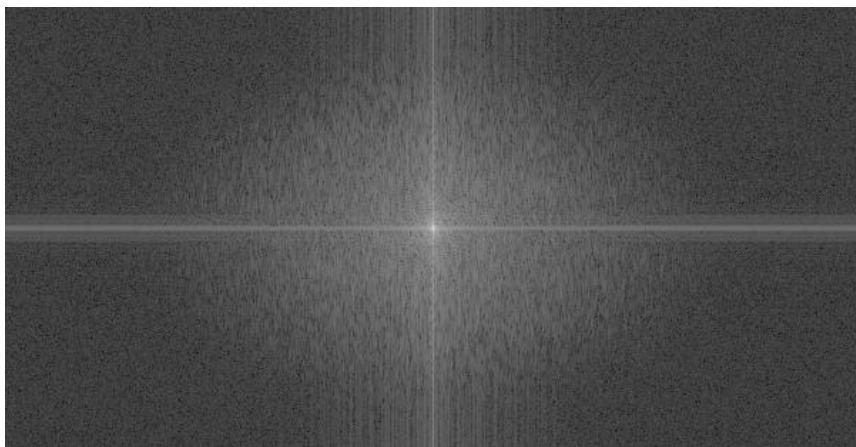


傅里叶反变换结果

2.4.3 傅里叶变换与卷积定理



原始图像



傅里叶变换结果

2.4.3 傅里叶变换与卷积定理

➤ 傅立叶变换的性质

- 线性 $F\{af_1(x, y) + bf_2(x, y)\} = aF_1(u, v) + bF_2(u, v)$
- 图像域原点平移 $F\{f(x-a, y-b)\} = F(u, v)e^{-2\pi i(au+bv)}$
- 频域原点平移 $F\{f(x, y)e^{2\pi i(u_0x+v_0y)}\} = F(u-u_0, v-v_0)$
- 对称性：若 $f(x, y)$ 是实数， $F(u, v)$ 关于原点共轭对称

$$F(u, v) = F^*(-u, -v)$$

➤ 卷积定理 (Convolution Theorem)

$$F\{f(x, y) * h(x, y)\} = F(u, v)H(u, v)$$

$$F\{f(x, y)h(x, y)\} = F(u, v) * H(u, v)$$

2.4.3 傅里叶变换与卷积定理

➤ 二维离散傅立叶变换(DFT)及其逆变换

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-2\pi i(xu + yv)} dx dy$$

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{2\pi i(xu + yv)} du dv$$



$$F(u, v) = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) e^{-2\pi i(um/M + vn/N)}$$

$$u = 0, 1, \dots, M-1$$

$$v = 0, 1, \dots, N-1$$

$$f(m, n) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{2\pi i(um/M + vn/N)}$$

$$m = 0, 1, \dots, M-1$$

$$n = 0, 1, \dots, N-1$$

$$F(u, v) = \frac{1}{M} \sum_{m=0}^{M-1} \left[\frac{1}{N} \sum_{n=0}^{N-1} f(m, n) e^{-2\pi i(vn/N)} \right] e^{-2\pi i(um/M)}$$

傅立叶变换 (Matlab)

- DFT和逆DFT都是周期无穷的，周期由M和N决定

$$F(u, v) = F(u + M, v) = F(u, v + N) = F(u + M, v + N)$$

$$f(m, n) = f(m + M, n) = f(m, n + N) = f(m + M, n + N)$$

- 二维DFT

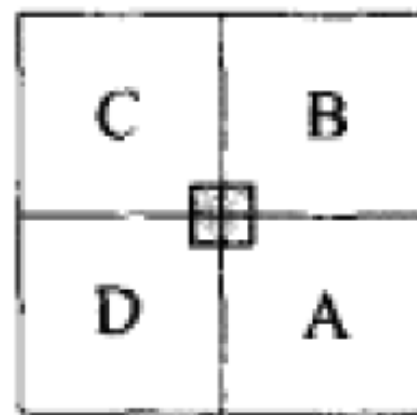
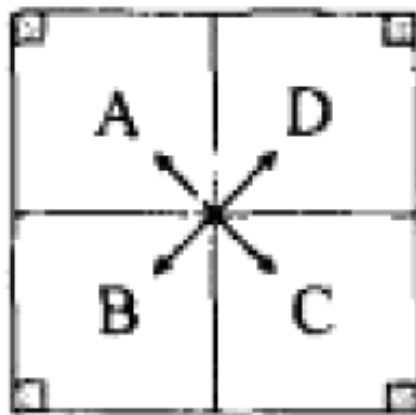
(matlab:fft2)

- 二维逆DFT

(matlab:ifft2)

- 重排数据

(matlab:fftshift)



Matlab

```
i = imread('miss.bmp');  
i = i(:,:,1);  
imshow(i,[]);
```

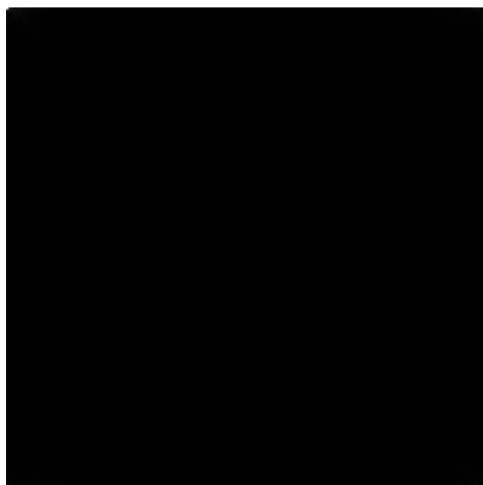
```
F=fft2(i);  
imshow(abs(F),[]);
```

```
Fs=fftshift(F);  
imshow(abs(Fs),[]);
```

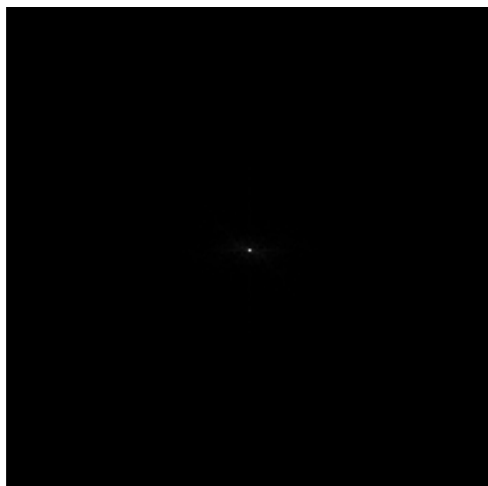
```
F1=log(1+abs(Fs));  
imshow(abs(F1),[]);
```



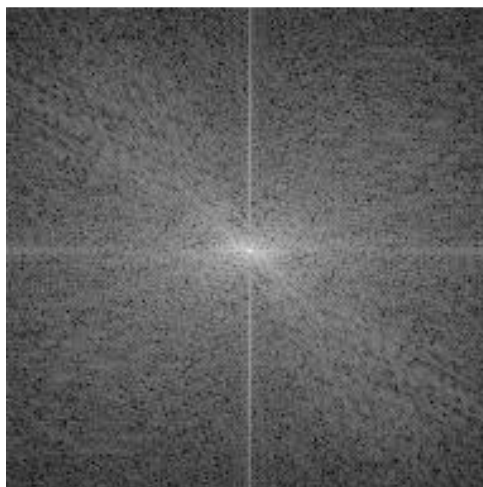
原始图像



DFT变换结果



fftshift结果



对数显示

傅立叶变换 (OpenCV)

➤ 离散傅立叶正/逆变换(OpenCV:dft)



- 根据输入输出数据判断一维、二维
- 通过函数参数控制正逆变换

➤ 程序要点

- 函数dft的输入输出是浮点数，而图像数据类型是8位无符号数，需要数据类型转换
- 通过函数(OpenCV:getOptimalDFTSize)获得最佳变换尺寸，使DFT可按照快速傅立叶变换(FFT)计算
- 变换结果分为实部和虚部，计算频谱
- 与matlab类似，需要进行数据重排（自行编程实现）

2.4.3 傅里叶变换与卷积定理

➤ 利用快速傅里叶变换(FFT)和卷积定理计算卷积

- 二维卷积的计算复杂度: $O(n^4)$
- 快速傅立叶变换复杂度: $O(n^2 \log^2 n)$
- 通过卷积定理建立卷积与其傅立叶变换的关系:

$$F\{f(x, y) * h(x, y)\} = F(u, v)H(u, v)$$

$$F\{f(x, y)h(x, y)\} = F(u, v) * H(u, v)$$

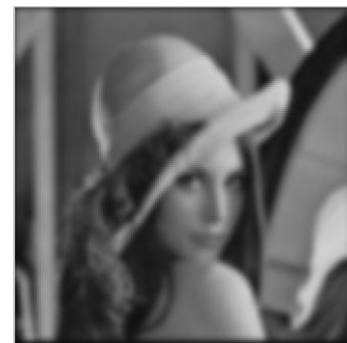
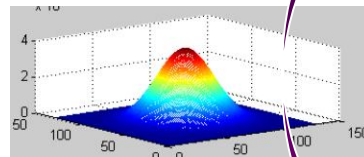
$$\begin{array}{c}
 g(x, y) \xleftarrow{O(n^4)} \begin{array}{c} f(x, y) \xrightarrow{O(n^2 \log^2 n)} F(u, v) \\ h(x, y) \xrightarrow{O(n^2 \log^2 n)} H(u, v) \end{array} \\
 \xrightarrow{O(n^2)} G(u, v) \xrightarrow{O(n^2 \log^2 n)} g(x, y)
 \end{array}$$

2.4.3 傅里叶变换与卷积定理

➤ 举例：利用FFT和卷积定理计算卷积

Matlab

```
img = imread('miss.bmp');
img = double(imresize(img(:,:,1),[2048,2048]));
h = fspecial('gaussian',[128 128],20);
% 直接使用二维卷积
g1 = uint8(conv2(img,h,'same'));
% 使用FFT和卷积定理
f_img = fft2(img);
f_h   = fft2(h,2048, 2048);
f_g   = f_img .* f_h;
g2 = uint8(ifft2(f_g));
```



时间已过 5.166931 秒。

时间已过 0.221810 秒。

2.5 图像分析中的数据结构

➤ 2.5.1 矩阵



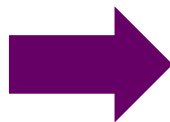
156	145	142	140	142	167	180	172	172	158	172	155	135	129	135	140	138	138	141
151	152	161	140	137	169	181	177	175	176	139	87	88	110	122	130	137	143	148
149	173	164	131	136	172	183	187	180	122	58	53	66	69	76	82	97	119	142
167	185	155	129	153	176	188	177	117	86	105	110	98	84	65	39	39	51	77
183	181	135	140	170	193	169	101	88	115	123	110	127	128	102	79	51	40	33
190	166	136	160	194	156	80	76	85	92	100	65	81	128	89	81	73	71	60
188	164	157	188	140	55	52	58	50	40	45	24	29	55	28	28	64	80	84
180	171	187	130	37	35	30	23	25	20	15	17	23	25	17	11	13	47	98
177	190	132	39	30	22	17	18	21	17	17	16	49	114	81	23	11	7	46
194	144	49	40	27	13	24	38	28	28	12	38	61	175	177	114	30	35	44
162	69	62	41	23	13	33	72	47	54	40	50	51	177	197	189	99	51	72
84	79	89	74	63	23	23	79	77	45	43	42	123	200	194	190	134	61	77
81	85	96	105	97	62	40	64	94	90	76	128	187	194	196	165	114	96	79
93	92	98	112	110	103	79	71	76	93	109	128	142	153	151	155	136	117	99
98	103	104	110	110	111	105	107	85	77	67	97	98	89	103	132	144	142	110
103	111	111	115	117	109	112	115	103	100	92	103	106	102	120	126	121	118	114
107	118	122	120	124	125	123	115	112	115	112	115	122	129	136	138	132	138	117
105	117	123	126	135	136	135	138	133	133	133	140	146	146	142	142	139	138	108
104	114	123	126	137	140	138	151	151	144	142	146	150	144	144	147	139	127	109

2.5 图像分析中的数据结构

➤ 2.5.1 矩阵



灰度图像



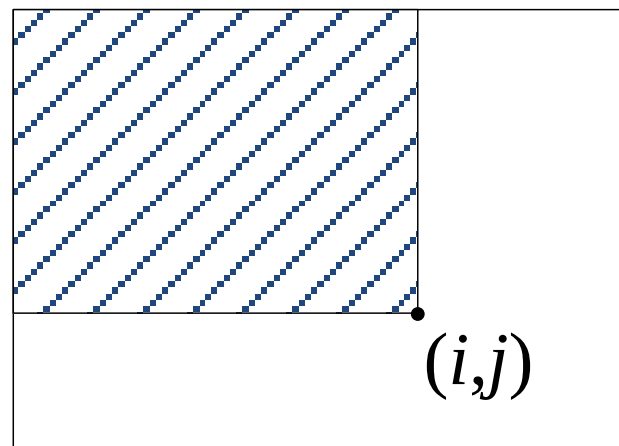
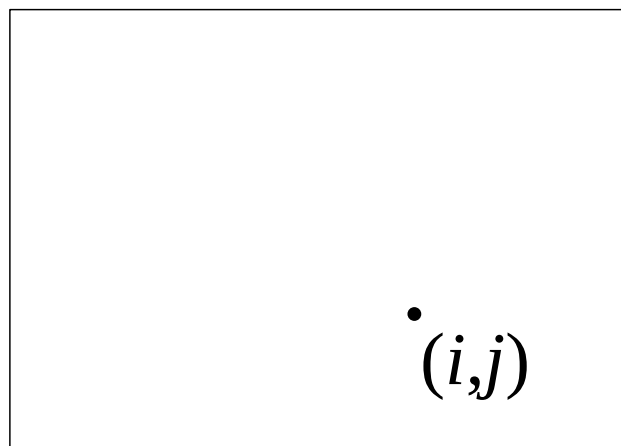
二值图像

2.5.1 矩阵

➤ 积分图像 (integral image)

- 一种描述图像全局信息的矩阵表示方法
- 对于原图像 f ，积分图像 ii 中的元素定义为：

$$ii(i, j) = \sum_{k \leq i, l \leq j} f(k, l)$$



2.5.1 矩阵

算法：积分图像构建

1. 用 $s(i, j)$ 表示行方向的累加和，初始化 $s(i, -1) = 0$
2. 用 $ii(i, j)$ 表示积分图像，初始化 $ii(-1, j) = 0$
3. 逐行扫描图像，递归使用下述公式计算每个像素 (i, j) 行方向的累加和 $s(i, j)$ ，积分图像 $ii(i, j)$ 的值

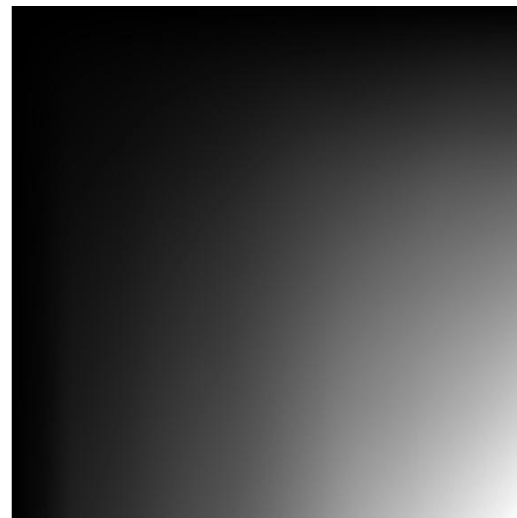
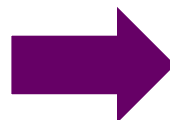
$$s(i, j) = s(i, j - 1) + f(i, j)$$

$$ii(i, j) = ii(i - 1, j) + s(i, j)$$

4. 扫描一遍图像，完成积分图像构建

2.5.1 矩阵

➤ 积分图像 (integral image)



2.5.1 矩阵

➤ 积分图像 (integral image)

- 利用积分图像计算任意区域内的像素和

点1的积分 $SAT1 = \text{Sum}(Ra)$

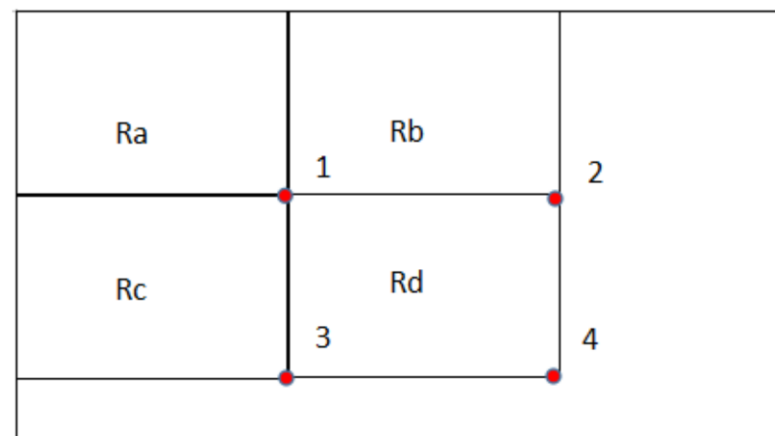
点2的积分 $SAT2 = \text{Sum}(Ra) + \text{Sum}(Rb)$

点3的积分 $SAT3 = \text{Sum}(Ra) + \text{Sum}(Rc)$

点4的积分 $SAT4 = \text{Sum}(Ra) + \text{Sum}(Rb) + \text{Sum}(Rc) + \text{Sum}(Rd)$

$\text{Sum}(Rd) = SAT4 - SAT2 - SAT3$

算法复杂度: $O(4)$



积分图举例

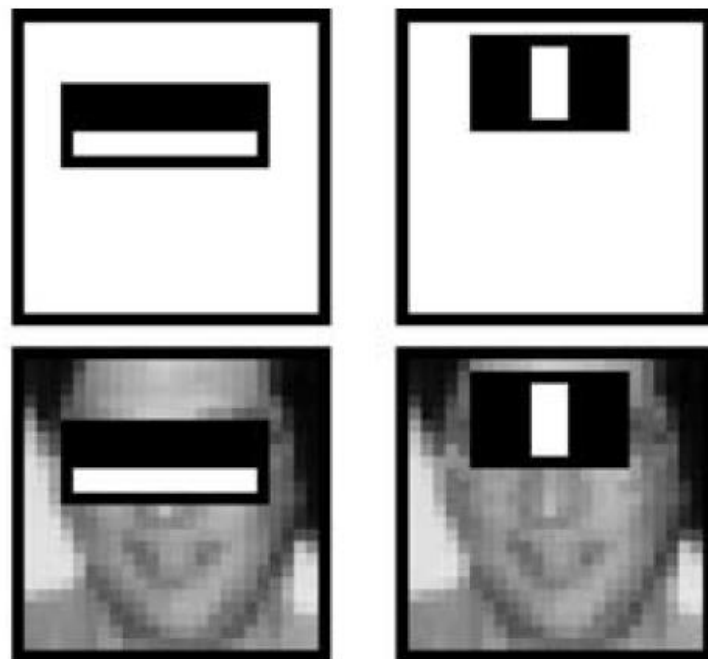
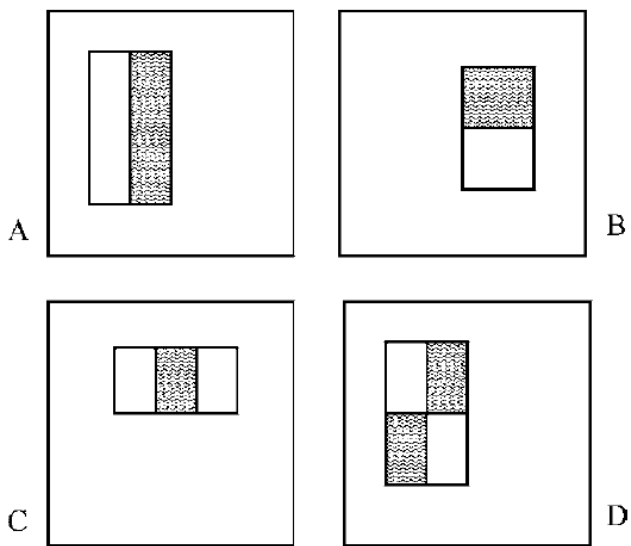
- VJ人脸检测
 - Harr特征与积分图
 - 特征选择+分类器融合
 - 级联结构



积分图举例

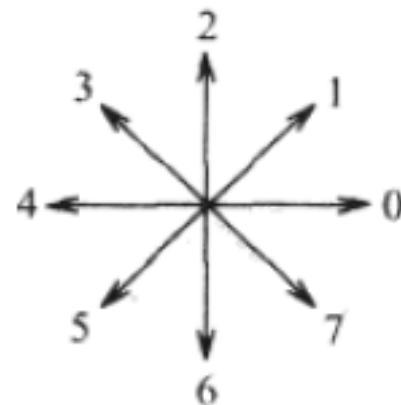
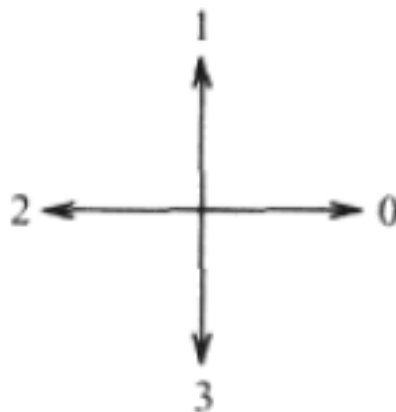
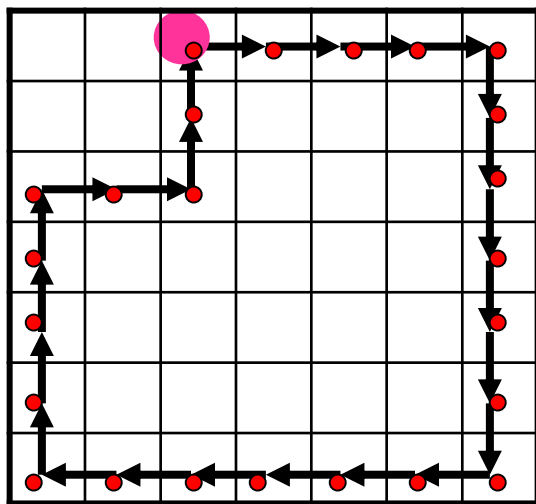
➤ 积分图像

- 2-矩形特征：使用6个点
- 3-矩形特征：使用8个点
- 4-矩形特征：使用9个点



2.5.2 链

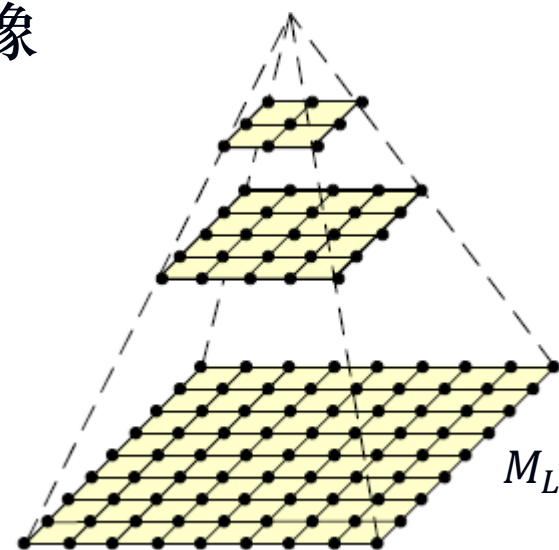
- 链在视觉算法中用于描述物体的边界
 - 链码 (chain codes)：一种边界的编码表示法
 - 边界由参考像素的坐标和符号序列来定义



4-链码: 000033333322222211110011

2.5.3 金字塔

- 金字塔：最简单的分层数据结构
- M型金字塔：矩阵型金字塔
 - 一个图像序列 $\{M_L, M_{L-1}, \dots, M_0\}$
 - M_L 是具有与原图像同样分辨率和元素的图像
 - M_{i-1} 是 M_i 降低一半分辨率得到的图像
 - 当需要对图像的不同分辨率进行处理时，分辨率每降低一层，数据量减少为原来的1/4，处理速度提高将近4倍

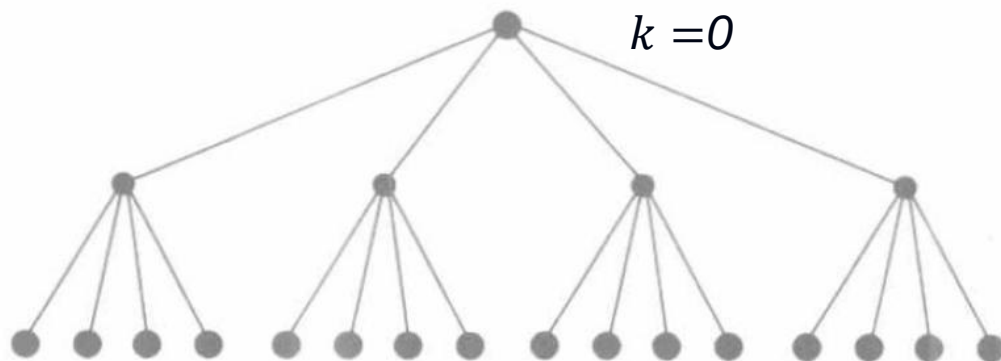


2.5.3 金字塔

- T型金字塔：树形金字塔，设 2^L 是原始图像大小
 - 一个结点集合

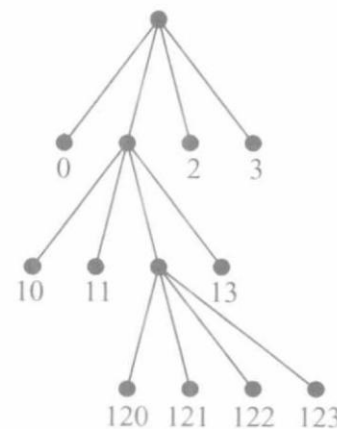
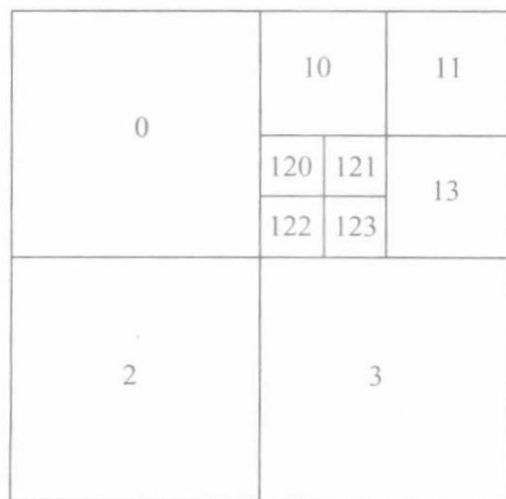
$$P = \{p = (k, i, j), k \in [0, L]; i, j \in [0, 2^k - 1]\}$$
 - 一个映射 F ，定义在金字塔结点 P_{k-1} 和 P_k 之间：

$$F(k, i, j) = (k - 1, i \div 2, j \div 2)$$
 - 一个函数 V ，将金字塔的结点 P 映射到 Z ，其中 Z 是对应于亮度级别数的所有数的子集， $Z = \{0, 1, 2, \dots, 255\}$



2.5.4 四叉树

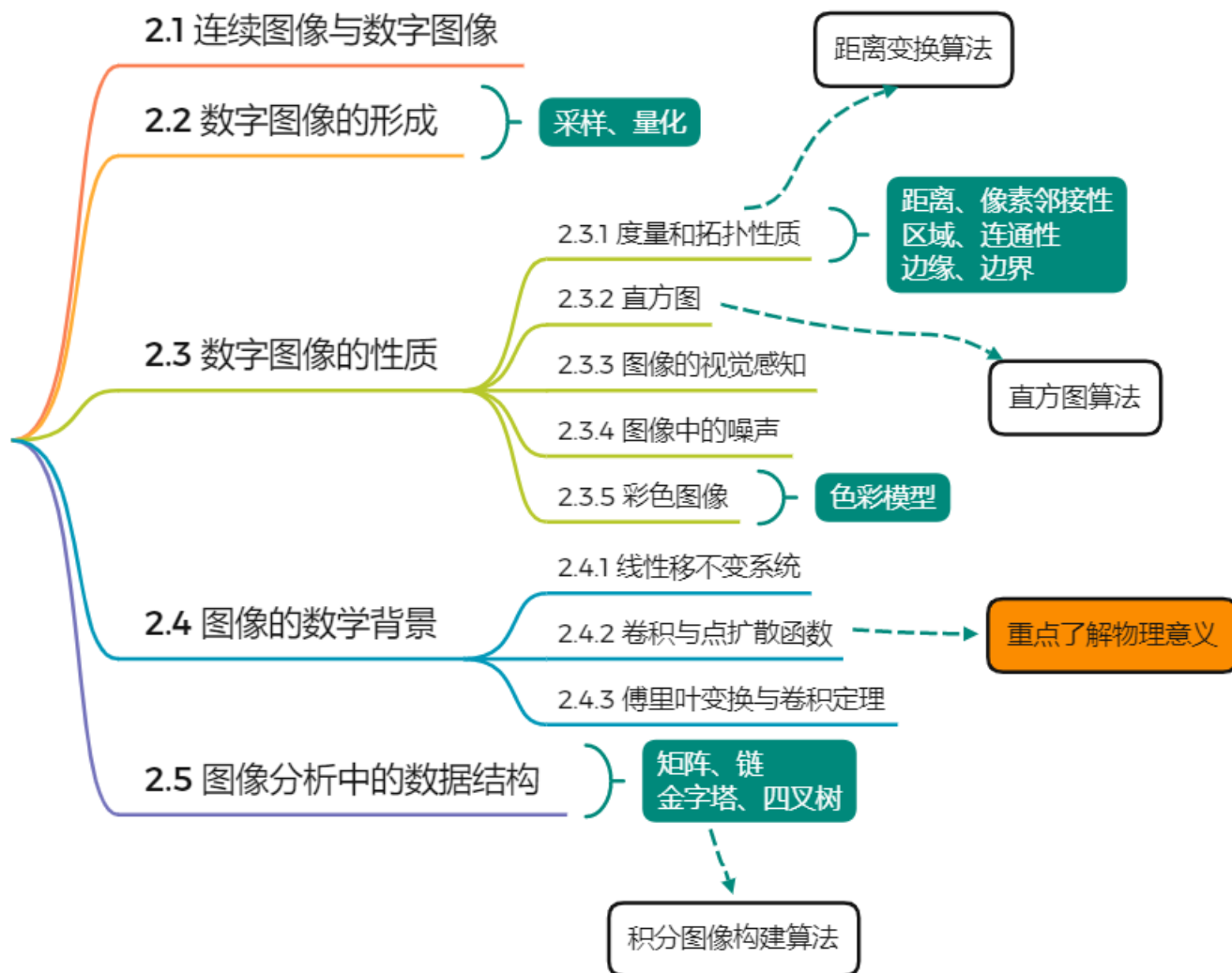
- 四叉树是对T型金字塔的改进，每个层次图像被分解为4个象限，无需在所有层次上保留结点
 - 优点：图像相加、计算面积、统计矩等操作算法简单
 - 缺点：依赖于物体位置、方向和大小，两个仅有微小差别的图像，可能会有完全不同的四叉树表示



总结



数字图像的概念与性质



The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學
Nankai University